Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Carthage
Institut Supérieur des Technologies de l'Information et de la Communication

Fascicule de Travaux Pratiques

Architecture IoT

3ème année SE-IoT

(Semestre 1)

Elaboré par :

Dr. Meriam Dhouibi

Dr. Afef Saidi

المكتبة
المعهد العالي
للتكنولوجيا المعلومات
و الإتصال
برج السدرية
جامعة قرطاج

Année universitaire 2025-2026

Ce contenu est mis à disposition selon les termes de la Licence Creative Commons Attribution 4.0 International (CC BY 4.0)

Plan

TP0 – Conception d'une solution IoT 78

TP1 – Présentation de la plateforme IoT ThingsBoard et mise en œuvre d'une application de supervision (température et humidité) 80

TP2 – Intégration d'un ESP32 et du capteur DHT11 avec ThingsBoard : contrôle d'actionneurs et visualisation des données 86

TP3 – Initiation à Node-RED : création de flux basiques, de communication via Telegram et d'un tableau de bord 99

TP4 – Mise en œuvre du protocole MQTT avec Node-RED pour la visualisation des données avec ESP32 et le capteur DHT11 116

TP5 – Communication HTTP avec Node-RED et ESP32 : implémentation des requêtes GET/POST et traitement des données 126

TP6 – Réception et traitement des requêtes HTTP POST via Wamp et Node-RED 135

Ce contenu est mis à disposition selon les termes de la Licence Creative Commons Attribution 4.0 International (CC BY 4.0)

TP0 – Conception d'une solution IoT

Introduction

Dans ce TP, nous allons concevoir une solution complète pour l'Internet des Objets (IoT) en utilisant une plateforme IoT.

Objectifs

Après avoir terminé ce TP, vous devriez être capable de :

- Sélectionner les capteurs IoT les plus adaptés à la solution IoT globale.
- Identifier les meilleures façons de connecter un dispositif IoT à la plateforme IoT.
- Expliquer comment les données envoyées depuis un dispositif IoT peuvent être gérées et utilisées par la plateforme IoT.

Manipulation

I. Partie 1 : Choix des dispositifs IoT

Répondez aux questions suivantes pour vous aider à choisir le dispositif IoT que vous utiliserez dans votre solution :

1. Quelle est la valeur ajoutée globale que votre solution est censée apporter ? (Par exemple : amélioration de l'efficacité énergétique, surveillance en temps réel, réduction des coûts, etc.)
2. Quelles sont les contraintes qui pourraient influencer le choix du dispositif ? (Par exemple : existe-t-il des contraintes de consommation énergétique ? Y a-t-il des problèmes de connectivité, comme l'indisponibilité de signaux sans fil ?)
3. À quelle fréquence le dispositif doit-il envoyer des données à la plateforme pour fournir des mesures efficaces au bon fonctionnement de la solution ? (Par exemple : toutes les secondes, toutes les minutes, à chaque événement, etc.)

II. Partie 2 : Spécification de la topologie réseau des dispositifs IoT

Dans cette partie, vous devez spécifier comment les dispositifs sont connectés à Internet :

1. Allez-vous connecter les dispositifs directement à la plateforme, par exemple en utilisant un réseau mobile ? Ou bien avez-vous besoin de connecter les dispositifs à une passerelle (gateway) pour qu'ils puissent accéder à la plateforme ? (Par exemple : connexion directe via 4G/5G, ou utilisation d'une passerelle locale avec un protocole de communication court portée.)
2. Quels protocoles de communication prévoyez-vous d'utiliser, par exemple Bluetooth Low Energy, WiFi, Zigbee, LoRa, etc. ? (Indiquez les protocoles en fonction des besoins de portée, de consommation énergétique, de débit, etc.)

III. Partie 3 : Explication de la gestion et de l'utilisation des données

Dans cette partie, décrivez comment les données sont stockées et mises à jour :

1. Comment les données sont-elles stockées sur la plateforme et à quelle fréquence sont-elles mises à jour ? (Par exemple : les données peuvent être stockées dans une base de données cloud liée à la plateforme IoT. Elles sont mises à jour en temps réel, à intervalles réguliers, ou uniquement lorsqu'un événement spécifique se produit. Précisez si des mécanismes de traitement ou de filtrage sont appliqués avant le stockage.)

IV. Partie 4 : Identification des types d'analytique pouvant être appliqués aux données collectées pour générer une meilleure valeur commerciale

Dans cette partie, identifiez les types d'analyses de données qui peuvent être appliqués aux données collectées. Vous pouvez envisager l'utilisation de l'analyse en continu (Streaming Analytics), de l'analyse prédictive (Predictive Analytics), ou tout autre type d'analyse pertinente pour votre solution.

1. Comment pouvez-vous utiliser les données collectées pour générer une meilleure valeur commerciale ?
   (Par exemple :
   - Utiliser l'analyse prédictive pour anticiper les pannes d'équipements et planifier la maintenance préventive.
   - Appliquer l'analyse en temps réel pour détecter des anomalies ou des comportements inhabituels immédiatement.
   - Utiliser l'analyse descriptive pour suivre les tendances, optimiser les processus ou ajuster les stratégies opérationnelles.
   - Mettre en place des tableaux de bord analytiques pour aider à la prise de décision basée sur les données collectées.)

TP1 – Présentation de la plateforme IoT ThingsBoard et mise en œuvre d'une application de supervision (température et humidité)

Introduction

ThingsBoard est une plateforme IoT open source offrant des fonctionnalités de gestion des appareils, de collecte de données, de traitement et de visualisation pour votre solution IoT. Dans ce TP, Vous allez apprendre comment intégrer ThingsBoard à un exemple de solution IoT.

Objectifs

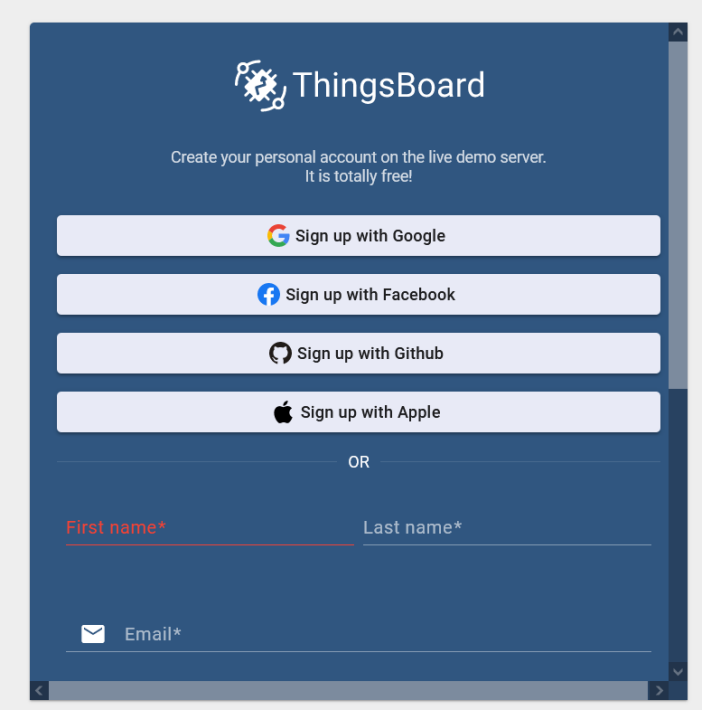
À la fin de ce TP vous serez en mesure de :

- Comprendre les bases des fonctionnalités et de l'architecture de ThingsBoard.
- Construire une application IoT avec ThingsBoard pour surveiller la température et l'humidité.
- Visualiser les données dans votre propre Dashboard.

Manipulation

1- Login

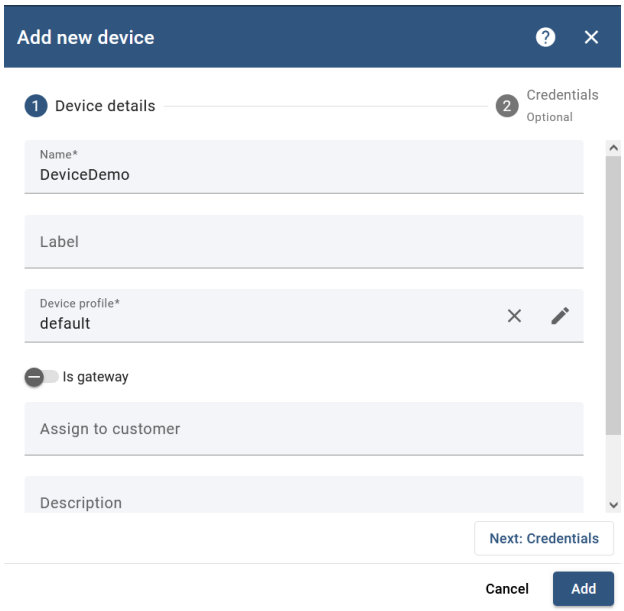
Pour accéder à la plateforme ThingsBoard, vous pouvez installer ThingsBoard sur votre ordinateur, ou bien vous pouvez utiliser « Live Demo »il suffit donc de créer un compte.



2- Création d'un device

Pour créer votre premier appareil connecté, vous devez :

1. Vous rendre sur l'onglet « Entities » puis « Devices ».
2. Cliquez ensuite sur le « + » en haut à droite puis « Add new device ».
3. Donnez-lui un nom et attribuez-lui un type.
4. Cliquez sur « Add ».

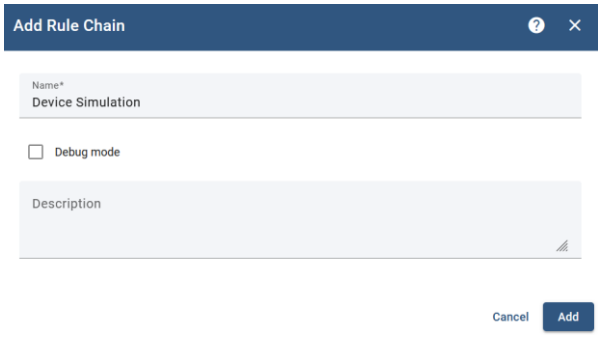


Votre device est désormais créé.

3- Simulation d'un device

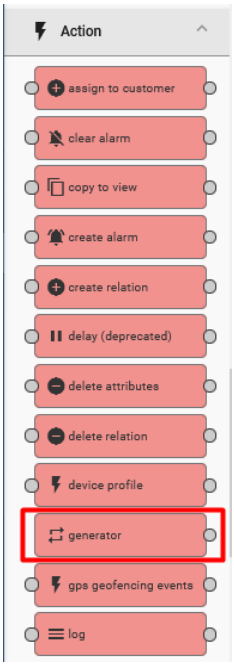
Afin de simplifier la démonstration, nous utiliserons dans ce TP un appareil connecté simulé. Pour simuler le senseur, nous allons utiliser le système de « Chaîne de règle » de ThingsBoard:

1. Cliquez sur l'onglet « Rule chains », puis créez une nouvelle chaîne de règles.
2. Donnez-lui un nom et cliquez sur « Add ».



Il faut désormais configurer cette chaîne de règles :

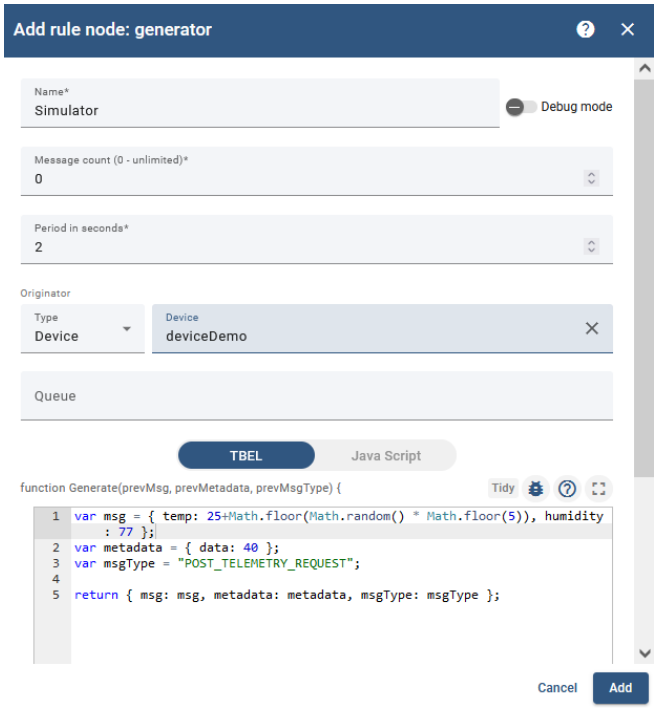
3. Pour simuler un device, nous allons avoir besoin d'un « Generator » : sélectionnez, dans la catégorie « Action », le bloc rouge nommé « Generator ».



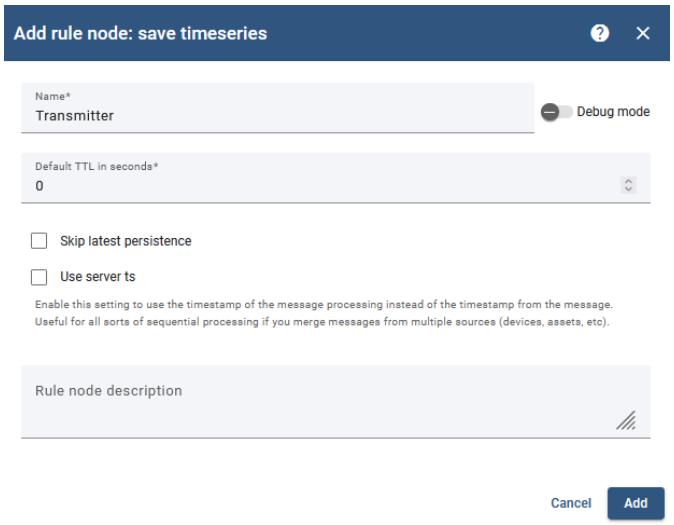
4. Dans son écran de configuration, attribuez-lui un nom.
5. Choisissez le nombre de secondes entre chaque message généré par le device. Dans notre exemple nous prendrons 2 secondes.
6. Sélectionnez ensuite « Device » comme origine et choisissez le device créé précédemment. Enfin, notre fonction de démonstration est la suivante :

```
var msg = { temp: 25+Math.floor(Math.random() * Math.floor(5)), humidity: 77
};
var metadata = { data: 40 };
var msgType = "POST_TELEMETRY_REQUEST";
return { msg: msg, metadata: metadata, msgType: msgType };
```

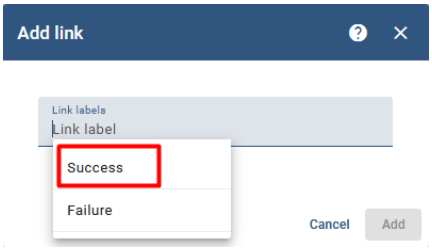
7. Cliquez ensuite sur « Add ».



8. Ensuite, sélectionnez dans la catégorie « Action », un bloc rouge « Save timeseries », ajoutez-le, nommez-le et cliquez sur « Add ». Ce bloc va vous permettre de stocker vos données.

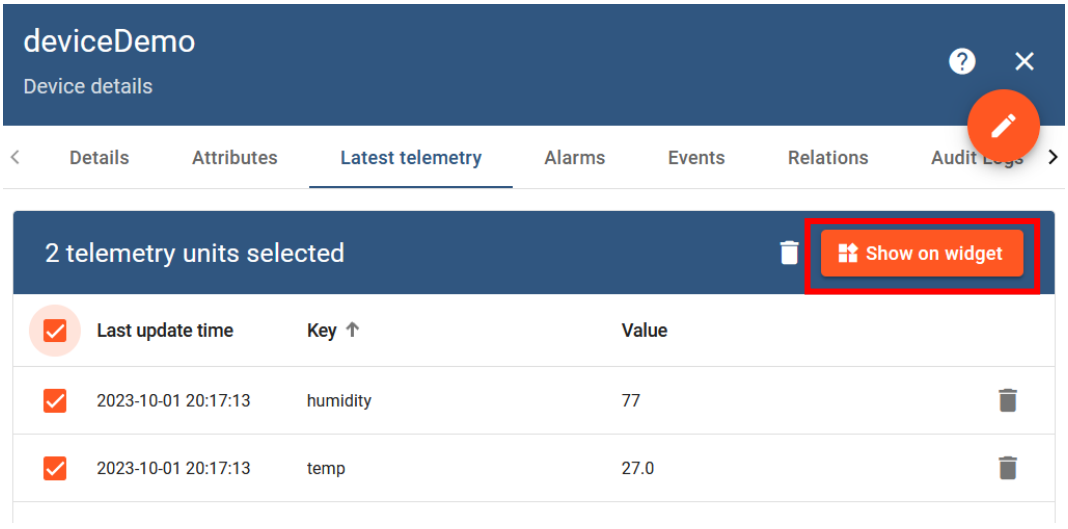


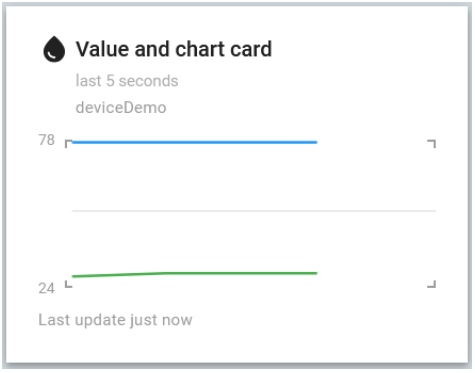
9. Reliez le bloc « Generator » au bloc « Save timeseries » en choisissant « Success » comme « Link lables » et cliquez sur « Add ». Ne vous préoccupez pas du bloc vert « Input ».



Votre device est maintenant simulé et vous pouvez le vérifier comme suit :

10. Cliquez sur l'onglet « Devices ».
11. Cliquez sur le device créé précédemment.
12. Cliquez sur l'onglet « Latest telemetry ». Vous pouvez désormais observer la température et l'humidité qui sont mises à jour toutes les deux secondes.





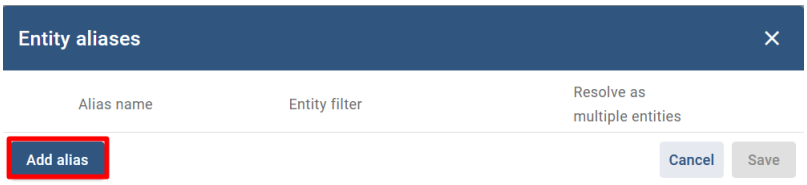
4- Visualisation des données dans le Dashboard

Maintenant que vous avez un appareil simulé opérationnel, vous allez pouvoir visualiser les informations qu'il renvoi sur un écran de contrôle appelé "Dashboard" grâce aux étapes suivantes :

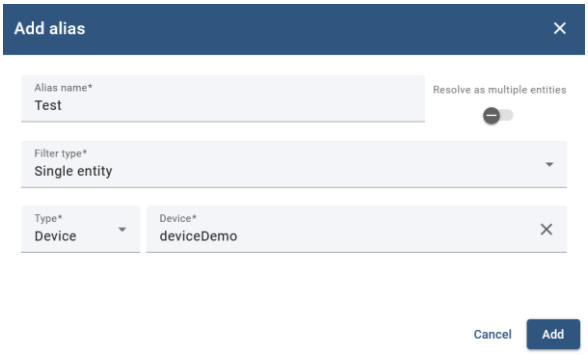
1. Cliquez sur l'onglet « Dashboards ».
2. Cliquez ensuite sur le « + » en haut à droite puis « Create new dashboard ».
3. Nommez-le et cliquez sur « Add ». Il est possible d'attribuer ce Dashboard à un utilisateur en cliquant sur l'icône « Manage assigned customers » .

Après avoir cliqué sur le Dashboard, il faut définir l'alias qui sera utilisé. Il s'agit d'un nom secondaire qui va être attribué à un certain device. Pour se faire :

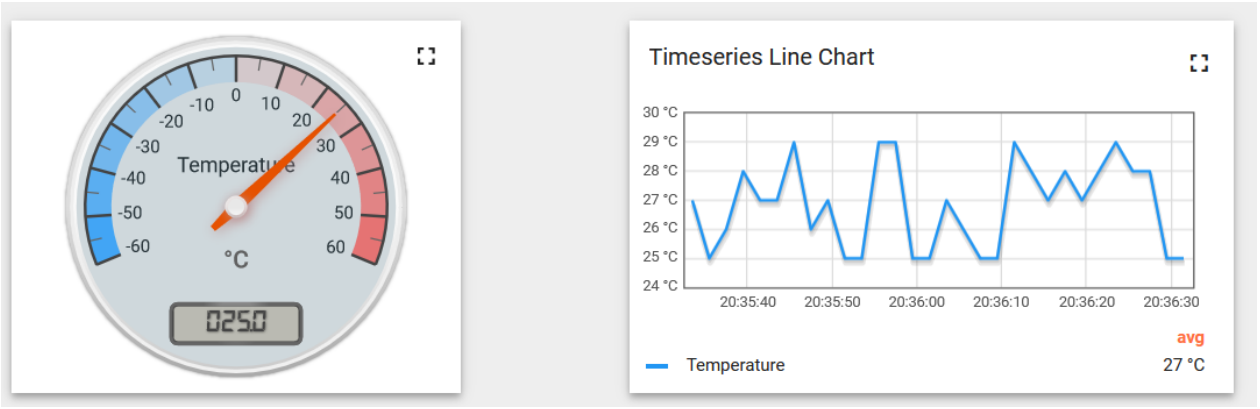
4. Cliquez ensuite sur l'icône « Entity Alias » en haut à droite. Vous pouvez alors ajouter votre alias en saisissant un nom.



5. Dans votre cas, sélectionnez le filtre « Single entity » puis le type « Device » avec le nom de votre device.



Votre alias est désormais créé. Vous pouvez désormais ajouter des éléments à votre Dashboard en cliquant sur « Add new widget » (une carte, une jauge de température, un graphique temporel...).



TP2 – Intégration d'un ESP32 et du capteur DHT11 avec ThingsBoard : contrôle d'actionneurs et visualisation des données

Introduction

ThingsBoard vous permet de surveiller et de contrôler les appareils IoT. Dans ce TP, vous allez apprendre comment intégrer ThingsBoard pour le contrôle des LED et la visualisation de température et d'humidité avec la carte de développement ESP32 et le capteur DHT11.

Objectifs

À la fin de ce TP vous serez en mesure de :

- Tester le bon fonctionnement des capteurs et de la carte de développement IoT utilisés.
- Utiliser la carte ESP32 pour récupérer les données issues de capteur DH11 et les envoyées à la plateforme ThingsBoard pour surveiller la température et l'humidité.
- Visualiser les données obtenues du capteur dans un Dashboard.
- Contrôler des LED à travers le Dashboard de la plateforme ThingsBoard.

Manipulation

I. Matériels utilisés

1. Carte de développement NodeMCU32-S1

ESP32 est une série de microcontrôleurs système sur puce à faible coût et à faible consommation avec Wi-Fi autonome intégré et Bluetooth bi-mode.
La carte ESP32 est développé par la société Espressif, c'est une carte de développement à faible coût dédié à l'internet des objets (IoT) et les applications embarquées.
Le module ESP32 est facile à prendre en main grâce à ses ressemblances avec la carte Arduino.
NodeMCU32-S1 est un module basé sur un ESP32 cadencé à 240 MHz et exécutant le firmware open source NodeMCU. Cette carte se programme via l'IDE Arduino et est compatible avec les scripts LUA.

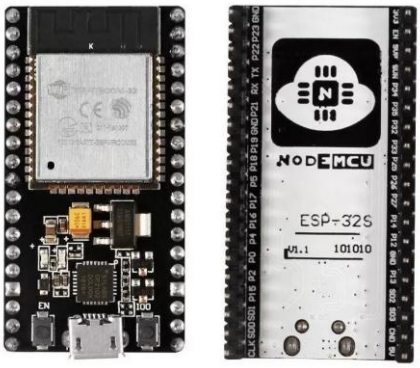
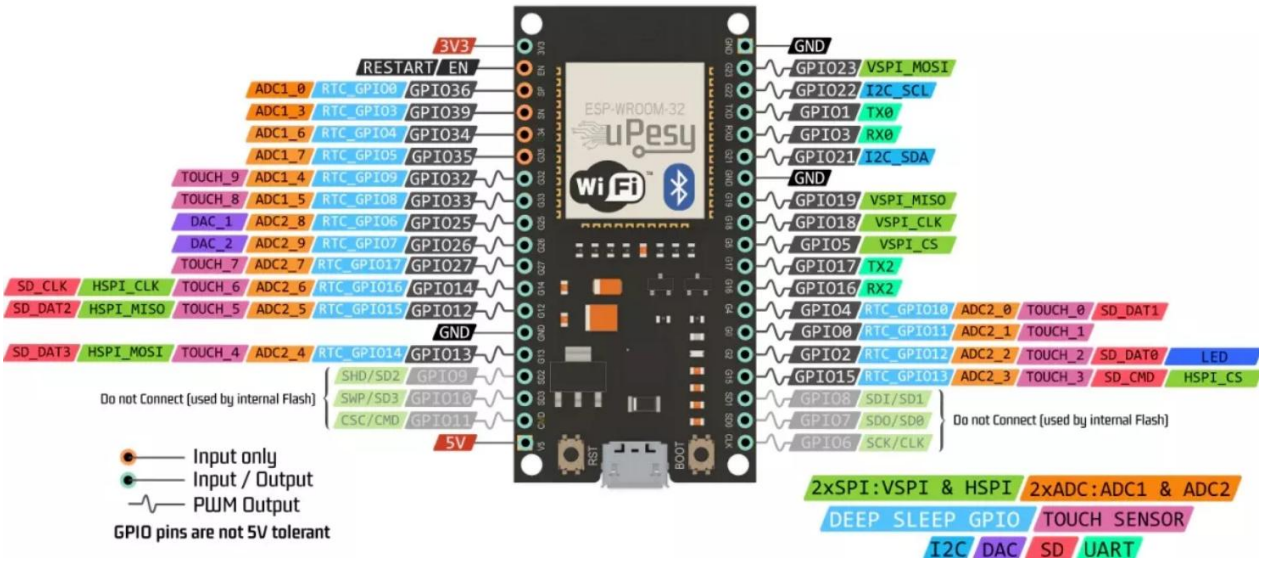
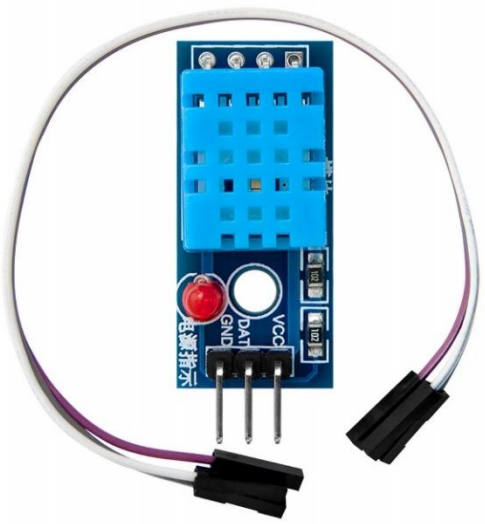


Schéma broches



2. Capteur de température et humidité DHT11



Le capteur DHT11 est capable de mesurer des températures de 0 à +50°C avec une précision de +/- 2°C et des taux d'humidité relative de 20 à 80% avec une précision de +/- 5%. Le module DHT11 fournit une sortie numérique proportionnelle à la température et à l'humidité mesurées par le capteur.
Le module DHT11 a 3 broches : +3.3V, GND et DATA. Les deux premières broches sont l'alimentation et la masse et elles sont utilisées pour alimenter le capteur, la troisième est le signal de sortie numérique du capteur.

3. **6 LED**
4. **6 résistances 330Ω**
5. **Plaque à essai**

II. Logiciels et services en ligne

- Arduino IDE
- ThingsBoard

III. Travail demandé

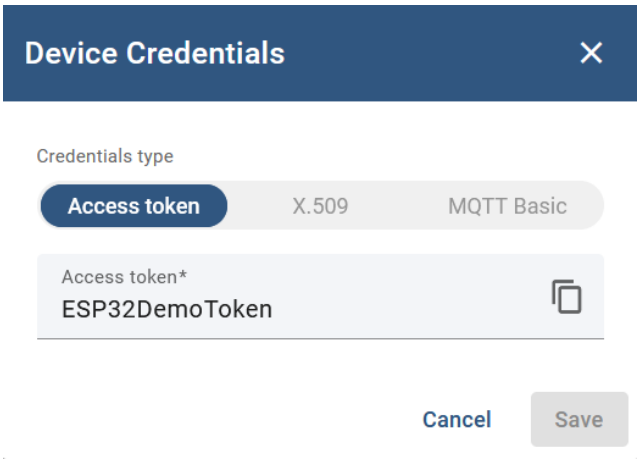
1. Configuration ThingsBoard

Cette étape contient les instructions nécessaires pour connecter votre appareil à ThingsBoard.

- Vous devez aller dans la section « Entities » puis « Devices ». Cliquez ensuite sur le bouton « + » et créez un appareil portant le nom « ESP32 Device ».

- Une fois le dispositif créé, ouvrez ses détails et cliquez sur « Manage credentials ».

- Copiez le jeton d'accès généré automatiquement à partir du champ « Access token ». Veuillez enregistrer ce jeton de dispositif. Il sera appelé plus tard $ACCESS_TOKEN.



- Téléchargez le fichier du Dashboard. Utilisez les instructions d'importation/exportation pour importer le Dashboard dans votre instance ThingsBoard.

2. Programmation de la carte NodeMCU32-S1

Le moyen le plus simple de programmer la carte NodeMCU32-S1 est d'utiliser l'IDE Arduino. Les sections suivantes décrivent cette approche.

a. Configuration de NodeMCU32-S1 et de l'IDE Arduino

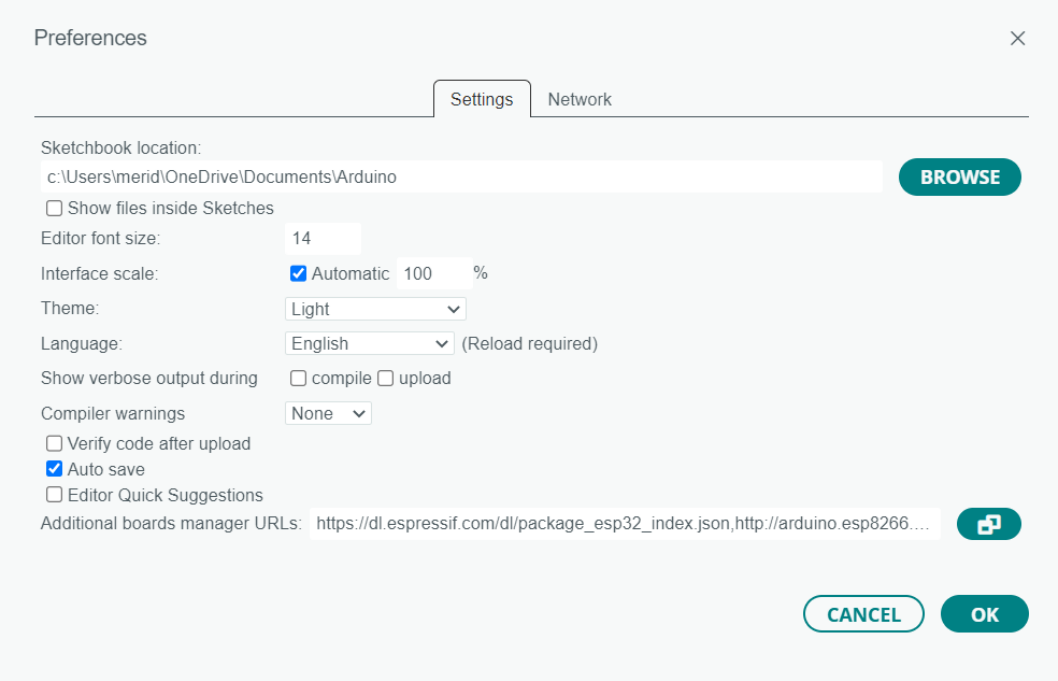
Vous aurez d'abord besoin de l'IDE Arduino et de tous les logiciels associés installés.

- Téléchargez et installez l'IDE Arduino.

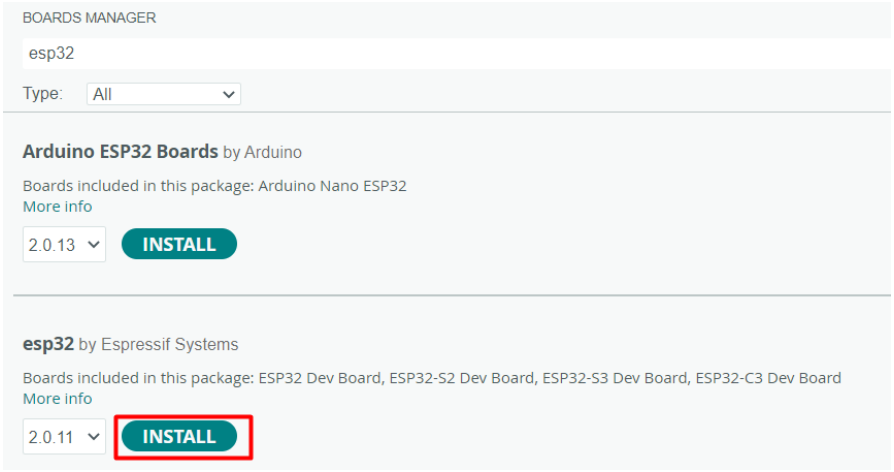
- La prise en charge de la carte NodeMCU32-S1 doit être ajoutée à l'IDE Arduino avant d'écrire le programme et le flasher dans la carte. Pour ce faire, installez-le package ESP32 comme décrit ci-dessous :

1- Ouvrez l'IDE Arduino.
2- Cliquez sur « Open File » puis « Preferences » et ajoutez les URL de gestionnaire de cartes dans le champ URL du gestionnaire de cartes supplémentaires, comme indiqué ci-dessous :

https://dl.espressif.com/dl/package_esp32_index.json,http://arduino.esp8266.com/stable/package_esp8266com_index.json



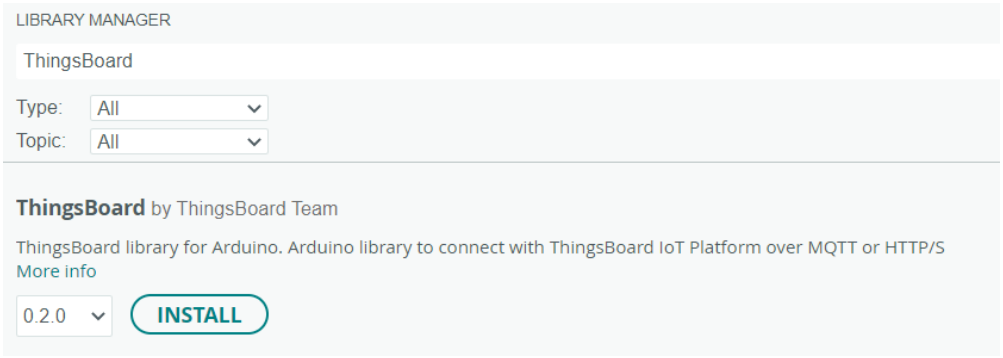
3- Sélectionnez « Tools » puis « Board » puis « Board manager menu ».
4- Tappez ESP32 dans le champ de recherche. Cliquez sur « Install ».



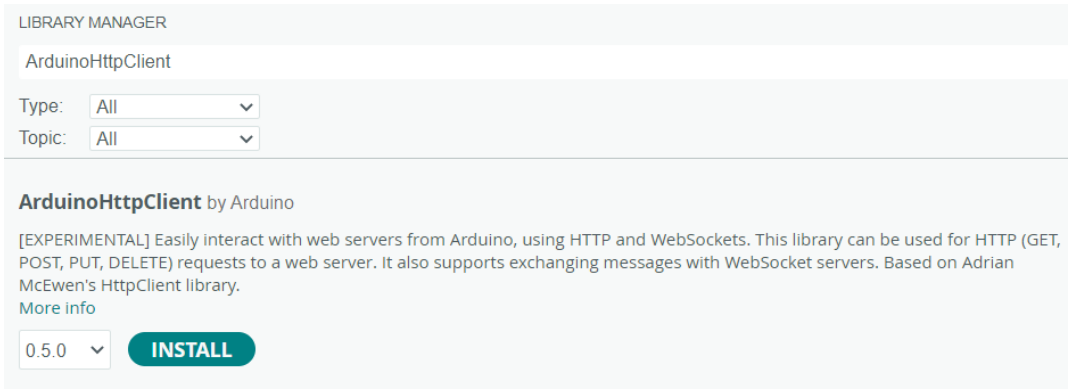
b. Installation du SDK Arduino ThingsBoard

Pour simplifier le développement d'applications, installez le SDK ThingsBoard Arduino et ses dépendances à partir du référentiel de bibliothèque Arduino standard :

1- Cliquez sur « Sketch » puis « Include Library ». Sélectionnez « Manage Libraries ».
2- Recherchez et installez les bibliothèques **ThingsBoard** Arduino SDK, **PubSubClient** de Nick O'Leary, **ArduinoHttpClient** et **ArduinoJSON**.





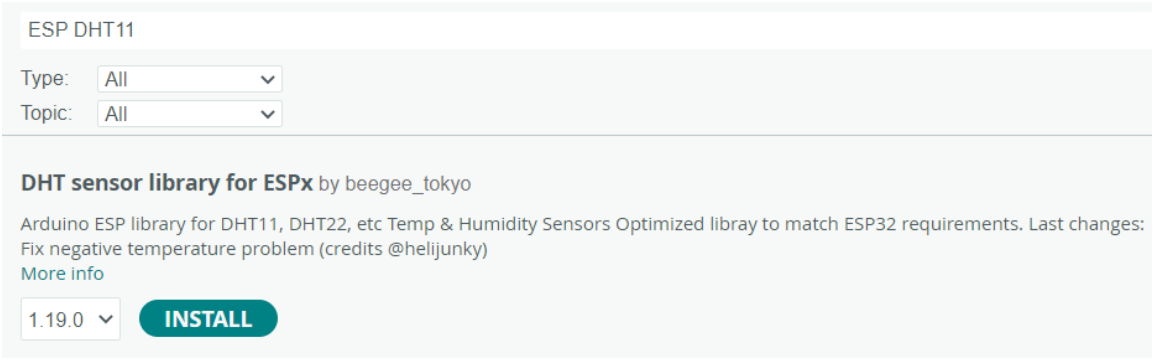




c. Installation de pilote ESP32 DHT11

Le capteur DHT11, connecté à la carte de développement NodeMCU32-S1 nécessite un driver. Pour l'installer, procédez comme suit :

1- Cliquez sur le menu « Sketch ». Ouvrez le sous-menu « Include Library ». Sélectionnez « Manage Libraries ».
2- Tapez « ESP DHT11 » dans le champ de recherche.
3- Cliquez sur installer « DHT Sensor Library for ESPx », comme indiqué ci-dessous :



d. Connexion de la carte NodeMCU32-S1 au PC

La carte NodeMCU32-S1 ne nécessite pas de connexion sophistiquée. Branchez simplement le câble micro-USB sur le PC et la carte.

Pour ceux qui ont un problème de connexion avec la carte vous devez installer ce driver.

e. Test des modules

Dans cette partie vous allez tester tous les composants :

1- Test carte

Ecrivez un code pour faire clignoter une LED par la carte ESP32 (NodeMCU32-S1) : éclairer et s'éteindre la LED alternativement à intervalles de 1 seconde. Vous pouvez utiliser l'exemple fourni par Arduino IDE.

2- Test capteur

Bronchez le capteur avec la carte comme suit :

VCC ->3.3V
GND -> GND
DATA -> P33

Ecrivez un code pour lire les valeurs issues du capteur. Vous pouvez utiliser l'exemple fourni par Arduino IDE.

3- Test Wi-Fi

Testez la connectivité du modèle Wi-Fi intégré dans la carte.

f. Brochage des LED

Brochez les LED à la carte comme suit.

Anode LED1 -> GPIO 25 avec une résistance de protection.
Anode LED2 -> GPIO 26 avec une résistance de protection.
Anode LED3 -> GPIO 32 avec une résistance de protection.
Anode LED4 -> GPIO 19 avec une résistance de protection.
Anode LED5 -> GPIO 22 avec une résistance de protection.
Anode LED6 -> GPIO 21 avec une résistance de protection.
Toutes les cathodes LED -> GND

g. Préparation du code

1- Vous devez modifier les constantes et variables suivantes dans le code.

- WIFI_AP - *nom de votre point d'accès*
- WIFI_PASSWORD - *mot de passe du point d'accès*
- TOKEN - *l'étape de configuration $ACCESS_TOKEN de ThingsBoard.*
- THINGSBOARD_SERVER - *Adresse HOST/IP ThingsBoard accessible au sein de votre réseau wifi. Spécifiez demo.thingsboard.io si vous utilisez un serveur live demo.*

2- Précisez le pin connecté au capteur DHT11 (pin DATA).

h. Programmation

Copiez le code, le compilez et le téléversez.

Code

```
#include <DHTesp.h>          // DHT for ESP32 library
#include <WiFi.h>            // WiFi control for ESP32
#include <ThingsBoard.h>     // ThingsBoard SDK

// Helper macro to calculate array size
#define COUNT_OF(x) ((sizeof(x)/sizeof(0[x])) / ((size_t)(!(sizeof(x) %
sizeof(0[x])))))
// WiFi access point
#define WIFI_AP_NAME         "*****"
// WiFi password
#define WIFI_PASSWORD        "****"

// See https://thingsboard.io/docs/getting-started-guides/helloworld/
// to understand how to obtain an access token
#define TOKEN "****"
// ThingsBoard server instance.
#define THINGSBOARD_SERVER "****"

// Baud rate for debug serial
#define SERIAL_DEBUG_BAUD    115200

// Initialize ThingsBoard client
WiFiClient espClient;
// Initialize ThingsBoard instance
ThingsBoard tb(espClient);
// the Wifi radio's status
int status = WL_IDLE_STATUS;

// Array with LEDs that should be lit up one by one
uint8_t leds_cycling[] = { 25, 26, 32 };
// Array with LEDs that should be controlled from ThingsBoard, one by one
uint8_t leds_control[] = { 19, 22, 21 };

// DHT object
DHTesp dht;
// ESP32 pin used to query DHT11
#define DHT_PIN ***

// Main application loop delay
int quant = 20;

// Initial period of LED cycling.
int led_delay = 1000;
// Period of sending a temperature/humidity data.
int send_delay = 2000;
```

```
// Time passed after LED was turned ON, milliseconds.
int led_passed = 0;
// Time passed after temperature/humidity data was sent, milliseconds.
int send_passed = 0;

// Set to true if application is subscribed for the RPC messages.
bool subscribed = false;
// LED number that is currenlty ON.
int current_led = 0;

// Processes function for RPC call "setValue"
// RPC_Data is a JSON variant, that can be queried using operator[]
// See https://arduinojson.org/v5/api/jsonvariant/subscript/ for more details
RPC_Response processDelayChange(const RPC_Data &data)
{
  Serial.println("Received the set delay RPC method");

  // Process data

  led_delay = data;

  Serial.print("Set new delay: ");
  Serial.println(led_delay);

  return RPC_Response(NULL, led_delay);
}

// Processes function for RPC call "getValue"
// RPC_Data is a JSON variant, that can be queried using operator[]
// See https://arduinojson.org/v5/api/jsonvariant/subscript/ for more details
RPC_Response processGetDelay(const RPC_Data &data)
{
  Serial.println("Received the get value method");

  return RPC_Response(NULL, led_delay);
}

// Processes function for RPC call "setGpioStatus"
// RPC_Data is a JSON variant, that can be queried using operator[]
// See https://arduinojson.org/v5/api/jsonvariant/subscript/ for more details
RPC_Response processSetGpioState(const RPC_Data &data)
{
  Serial.println("Received the set GPIO RPC method");

  int pin = data["pin"];
  bool enabled = data["enabled"];
```

```
  if (pin < COUNT_OF(leds_control)) {
    Serial.print("Setting LED ");
    Serial.print(pin);
    Serial.print(" to state ");
    Serial.println(enabled);

    digitalWrite(leds_control[pin], enabled);
  }

  return RPC_Response(data["pin"], (bool)data["enabled"]);
}

// RPC handlers
RPC_Callback callbacks[] = {
  { "setValue",         processDelayChange },
  { "getValue",         processGetDelay },
  { "setGpioStatus",    processSetGpioState },
};

// Setup an application
void setup() {
  // Initialize serial for debugging
  Serial.begin(SERIAL_DEBUG_BAUD);
  WiFi.begin(WIFI_AP_NAME, WIFI_PASSWORD);
  InitWiFi();

  // Pinconfig

  for (size_t i = 0; i < COUNT_OF(leds_cycling); ++i) {
    pinMode(leds_cycling[i], OUTPUT);
  }

  for (size_t i = 0; i < COUNT_OF(leds_control); ++i) {
    pinMode(leds_control[i], OUTPUT);
  }

  // Initialize temperature sensor
  dht.setup(DHT_PIN, DHTesp::DHT11);
}

// Main application loop
void loop() {
  delay(quant);

  led_passed += quant;
  send_passed += quant;
```

```
// Check if next LED should be lit up
if (led_passed > led_delay) {
  // Turn off current LED
  digitalWrite(leds_cycling[current_led], LOW);
  led_passed = 0;
  current_led = current_led >= 2 ? 0 : (current_led + 1);
  // Turn on next LED in a row
  digitalWrite(leds_cycling[current_led], HIGH);
}

// Reconnect to WiFi, if needed
if (WiFi.status() != WL_CONNECTED) {
  reconnect();
  return;
}

// Reconnect to ThingsBoard, if needed
if (!tb.connected()) {
  subscribed = false;

  // Connect to the ThingsBoard
  Serial.print("Connecting to: ");
  Serial.print(THINGSBOARD_SERVER);
  Serial.print(" with token ");
  Serial.println(TOKEN);
  if (!tb.connect(THINGSBOARD_SERVER, TOKEN)) {
    Serial.println("Failed to connect");
    return;
  }
}

// Subscribe for RPC, if needed
if (!subscribed) {
  Serial.println("Subscribing for RPC...");

  // Perform a subscription. All consequent data processing will happen in
  // callbacks as denoted by callbacks[] array.
  if (!tb.RPC_Subscribe(callbacks, COUNT_OF(callbacks))) {
    Serial.println("Failed to subscribe for RPC");
    return;
  }

  Serial.println("Subscribe done");
  subscribed = true;
}
```

```
  // Check if it is a time to send DHT11 temperature and humidity
  if (send_passed > send_delay) {
    Serial.println("Sending data...");

    // Uploads new telemetry to ThingsBoard using MQTT.
    // See https://thingsboard.io/docs/reference/mqtt-api/#telemetry-upload-
api
    // for more details

    TempAndHumidity lastValues = dht.getTempAndHumidity();
    if (isnan(lastValues.humidity) || isnan(lastValues.temperature)) {
      Serial.println("Failed to read from DHT sensor!");
    } else {
      tb.sendTelemetryFloat("temperature", lastValues.temperature);
      tb.sendTelemetryFloat("humidity", lastValues.humidity);
    }

    send_passed = 0;
  }

  // Process messages
  tb.loop();
}
void InitWiFi()
{
  Serial.println("Connecting to AP ...");
  // attempt to connect to WiFi network

  WiFi.begin(WIFI_AP_NAME, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Connected to AP");
}
void reconnect() {
  // Loop until we're reconnected
  status = WiFi.status();
  if ( status != WL_CONNECTED) {
    WiFi.begin(WIFI_AP_NAME, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
      delay(500);
      Serial.print(".");
    }
    Serial.println("Connected to AP");
  }
}
```

3- Contrôle des LED et visualisation des résultats

a. Visualisation des données et contrôle GPIO

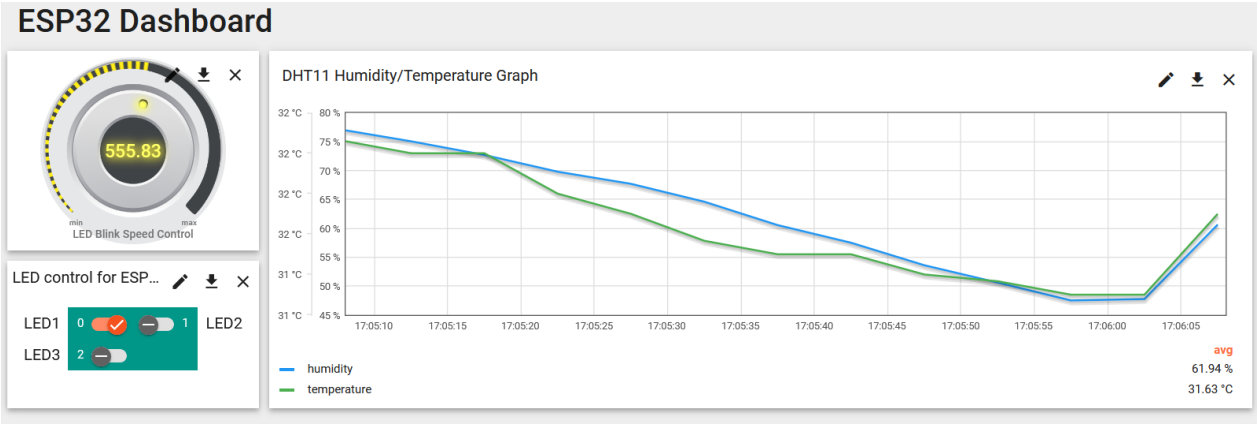
Enfin, connectez-vous à la plateforme ThingsBoard. Cliquez sur la section « Devices » et localisez « ESP32 Device », ouvrez les détails de dispositif et passez à l'onglet « Latest telemetry ». Si tout est configuré correctement, vous devriez pouvoir voir les dernières valeurs de « température » et « humidité » dans le tableau.

Ensuite, ouvrez la section « Dashboard », puis localisez et ouvrez « ESP32 Dashboard ». En conséquence, vous verrez un « time-series chart » affichant le niveau de température et d'humidité.

Vous devez également observer un contrôle GPIO pour votre dispositif. Il se compose de deux widgets : l'un sert à contrôler la vitesse de clignotement des 3 premiers LED (en millisecondes) et le second à allumer et éteindre des 3 derniers LED individuellement.

Vous pouvez changer l'état des GPIO à l'aide du « control panel ». Pour contrôler la vitesse de clignotement des LED, tournez simplement un bouton et observez un changement de vitesse.

Votre Dashboard doit apparaitre comme la figure illustrée ci-dessous.



b. Changement de fréquence d'envoi des données

Dans cette partie on vous demande de changer la fréquence d'envoi de données issues du capteur DHT11 à 15 secondes au lieu de 2 secondes.

TP3 – Initiation à Node-RED : création de flux basiques, de communication via Telegram et d'un tableau de bord

Introduction

Node-RED est un outil de programmation permettant de relier des périphériques matériels, des API et des services en ligne de manière nouvelle et intéressante.
Il fournit un éditeur basé sur un navigateur qui facilite la connexion des flux à l'aide du large éventail de nœuds de la palette qui peuvent être déployés dans son environnement d'exécution en un seul clic. Dans ce TP, vous allez explorer Node-RED et les différents nœuds.

Objectifs

À la fin de ce TP vous serez en mesure de :

- Comprendre les fonctionnalités des nœuds de base et construire des flux simples pour les déployer.
- Apprendre à créer un Dashboard.
- Utiliser les nœuds de Telegram pour établir une communication avec les utilisateurs des applications IoT.

Manipulation

I. Node-RED en local sous Windows

Node-RED utilise Node.js, un environnement d'exécution open-source et multiplateforme et backend pour exécuter le code JavaScript. Node.js est comme le moteur Node-RED.
Afin de gérer les nouveaux modules (extensions), Node.js utilise npm (Node Package Manager) qui est déjà intégré à Node.js.

1. Installation de Node.js et Node-RED

- Téléchargez et installez Node.js.
- Une fois installé, ouvrez une invite de commande et exécutez la commande suivante pour vous assurer que Node.js et npm sont correctement installés.

```
C:\WINDOWS\system32>node --version && npm --version
v14.18.0
6.14.15
```

- Exécutez la commande suivante pour installer Node-RED.

```
C:\WINDOWS\system32>npm install -g --unsafe-perm node-red
```

2. Exécution de Node-RED

- Pour exécuter Node-RED, utilisez la commande **node-red** dans une invite de commande. Vous devez garder le terminal ouvert pour que Node-RED continue de fonctionner.
- Utilisez l'adresse IP du serveur Node-RED pour ouvrir l'éditeur dans un navigateur Web.

```
26 Oct 10:24:36 - [info] Node-RED version: v3.1.0
26 Oct 10:24:36 - [info] Node.js  version: v18.18.2
26 Oct 10:24:36 - [info] Windows_NT 10.0.19045 x64 LE
26 Oct 10:24:45 - [info] Loading palette nodes
26 Oct 10:25:05 - [info] Dashboard version 3.6.1 started at /ui
26 Oct 10:25:08 - [info] Settings file  : C:\Users\saidi\.node-red\settings.js
26 Oct 10:25:08 - [info] Context store  : 'default' [module=memory]
26 Oct 10:25:08 - [info] User directory : \Users\saidi\.node-red
26 Oct 10:25:08 - [warn] Projects disabled : editorTheme.projects.enabled=false
26 Oct 10:25:08 - [info] Flows file     : \Users\saidi\.node-red\flows.json
26 Oct 10:25:08 - [warn]

---------------------------------------------------------------------
Your flow credentials file is encrypted using a system-generated key.

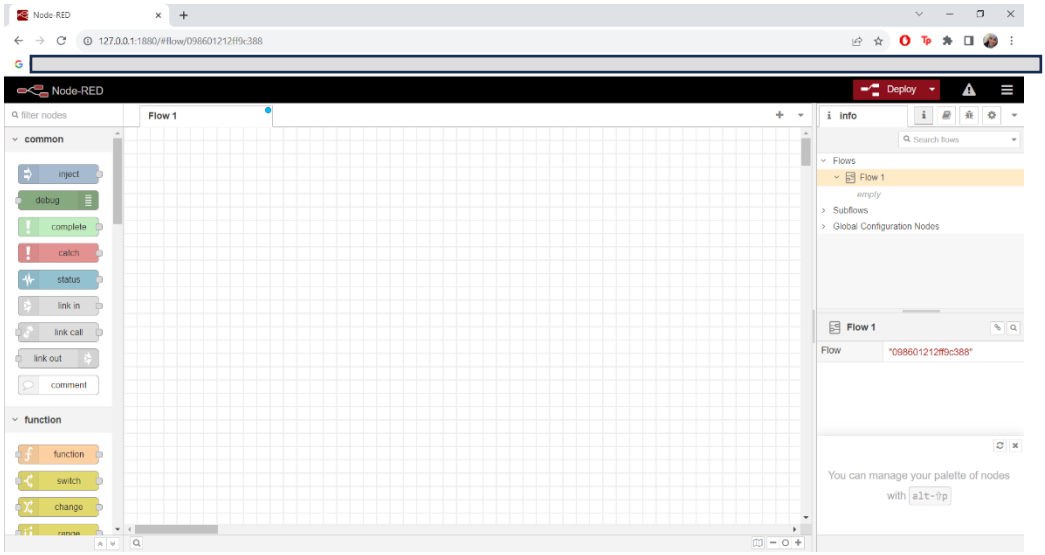
If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
---------------------------------------------------------------------

26 Oct 10:25:08 - [warn] Encrypted credentials not found
26 Oct 10:25:08 - [info] Waiting for missing types to be registered:
26 Oct 10:25:08 - [info]  - pac-device
26 Oct 10:25:08 - [info] Server now running at http://127.0.0.1:1880/
```

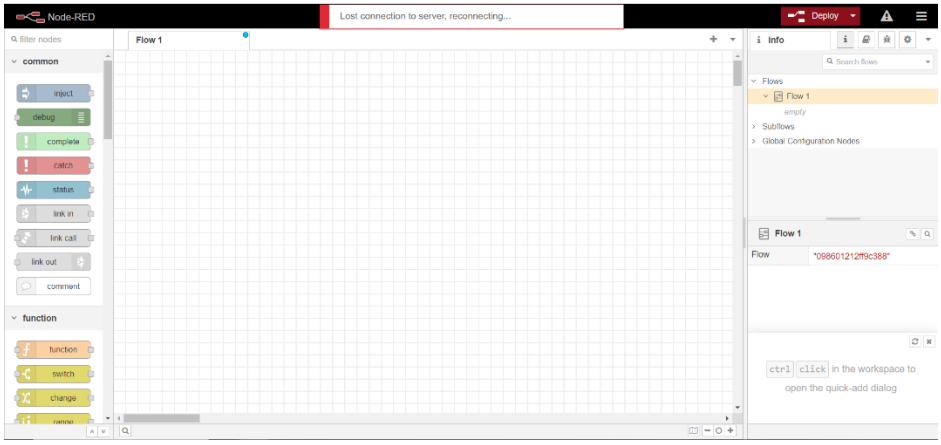
3. Accès à l'éditeur Node-RED

- Si vous utilisez un navigateur sur le même ordinateur qui exécute Node-RED, vous pouvez y accéder avec l'url : http://localhost:1880.
- Si vous utilisez un navigateur sur un autre ordinateur, vous devrez utiliser l'adresse IP de l'ordinateur exécutant Node-RED : http://<adresse IP> :1880.



Si nous voulons arrêter l'application, nous revenons à la fenêtre cmd et arrêtons le processus avec un « Ctrl + C ».

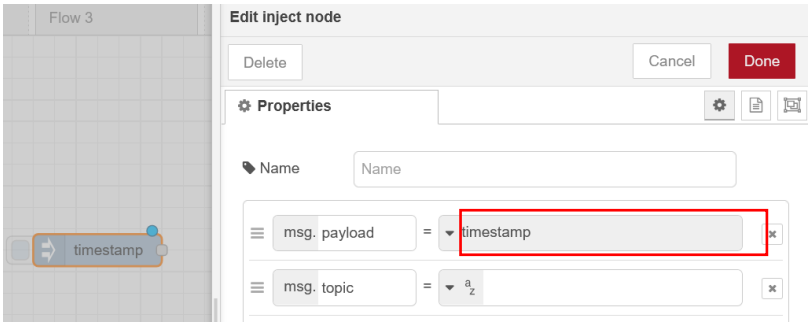
Le problème de déconnexion est alors mis en évidence dans la page de l'éditeur Node-RED associé. Pour se reconnecter, il suffit de relancer l'application Node-RED (commande : « node-red »).



II. Création des flux basiques Node-RED

Le nœud Inject : permet d'injecter des messages dans un flux, soit en cliquant sur le bouton du nœud, soit en définissant un intervalle de temps entre les injections.

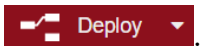
- Ajoutez ce nœud à l'espace de travail depuis la palette.
- Sélectionnez le nœud ajouté pour afficher des informations sur ses propriétés.



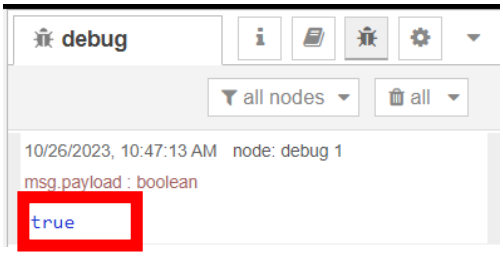
Le nœud d'injection fournit une donnée (un message de charge utile « payload message » qui est le contenu du message) :

- Modifiez le type de l'injection en booléen « Boolean ».
- Cliquez sur « Done ».

Le nœud Debug : fournit un moyen simple d'afficher les messages dans le volet de débogage.

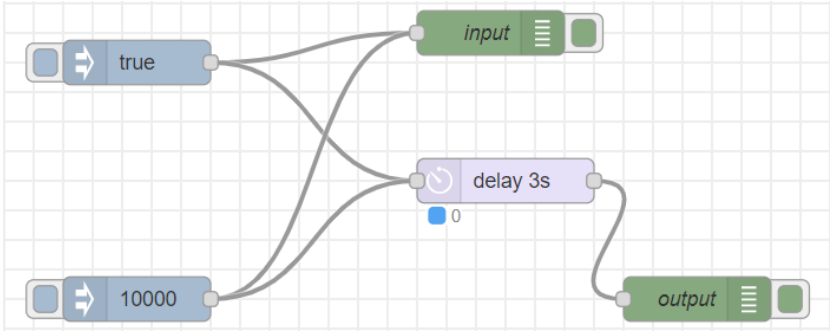
- Ajoutez ce nœud à l'espace de travail.
- Connectez les nœuds ensemble.
- Cliquez sur déployer 

.
- Injectez manuellement les données par un simple clic sur le rectangle à gauche du nœud d'injection  .
- Le message s'affiche ensuite dans la fenêtre de débogage.



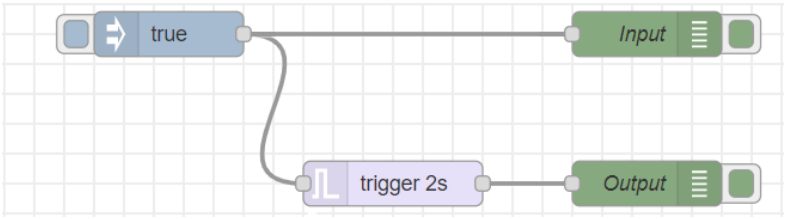
1. **Flux 1**
   - Ajoutez le nœud « **Delay** » sur l'espace de travail et reproduisez ce flux.

   Q1 : Expliquez les résultats obtenus dans la fenêtre de débogage.



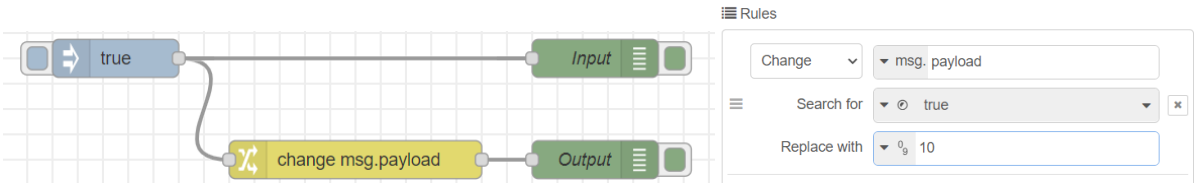
2. **Flux 2**
   - Ajoutez le nœud « **Trigger** » sur l'espace de travail et reproduisez le flux suivant.

   Q2 : Expliquez les résultats obtenus dans la fenêtre de débogage.



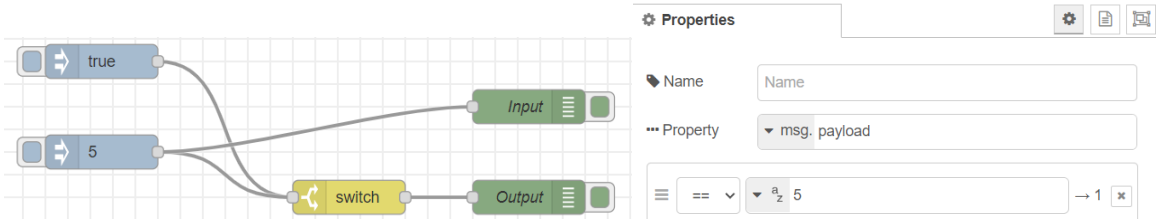
3. **Flux 3**
   - Ajoutez le nœud « **Change** » sur l'espace de travail et reproduisez le flux suivant. On vous donne les configurations à faire.

   Q3 : Expliquez les résultats obtenus dans la fenêtre de débogage.



4. **Flux 4**
   - Ajoutez le nœud « **Switch** » sur l'espace de travail et reproduisez le flux suivant. On vous donne les configurations à faire.

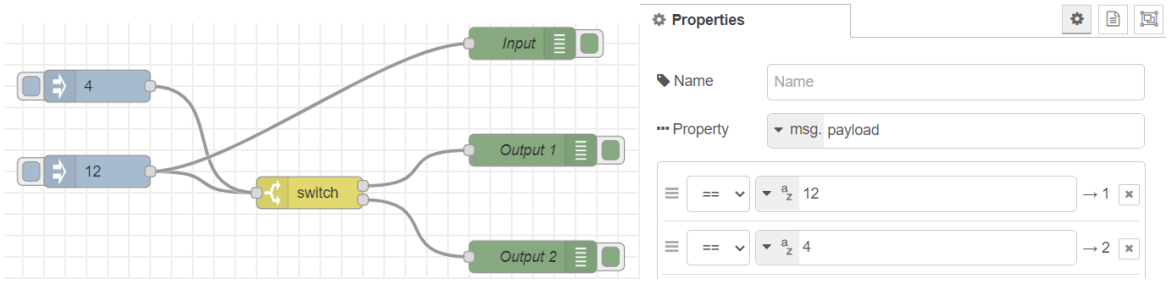
   Q4 : Expliquez les résultats obtenus.



5. Flux 5

- Reproduisez le flux suivant. On vous donne les configurations à faire.

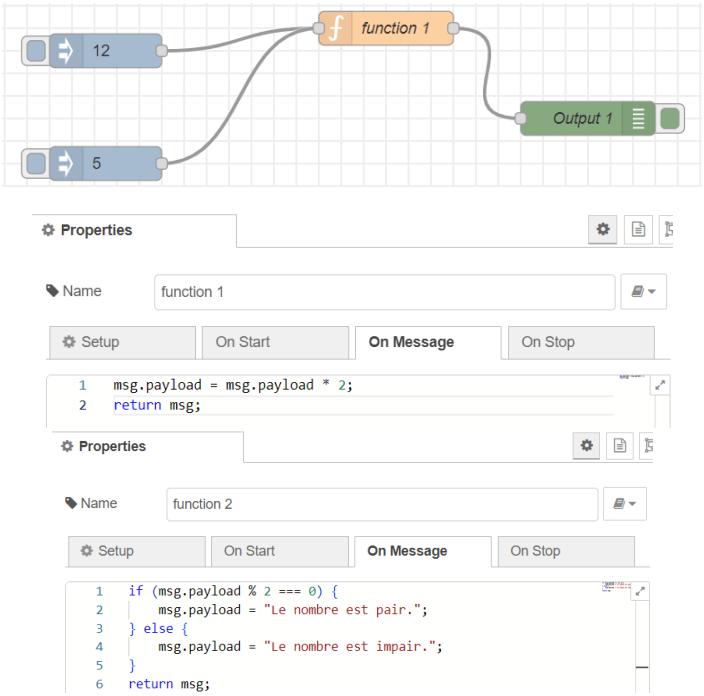
Q5 : Expliquez les résultats obtenus.



6. Flux 6

- Ajoutez le nœud « **Function** » sur l'espace de travail et reproduisez le flux suivant avec les deux exemples de fonctions. On vous donne les codes JS.

Q6 : Expliquez les résultats obtenus.

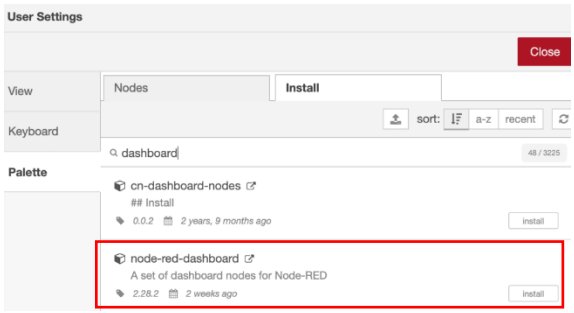


II. Création de Dashboard

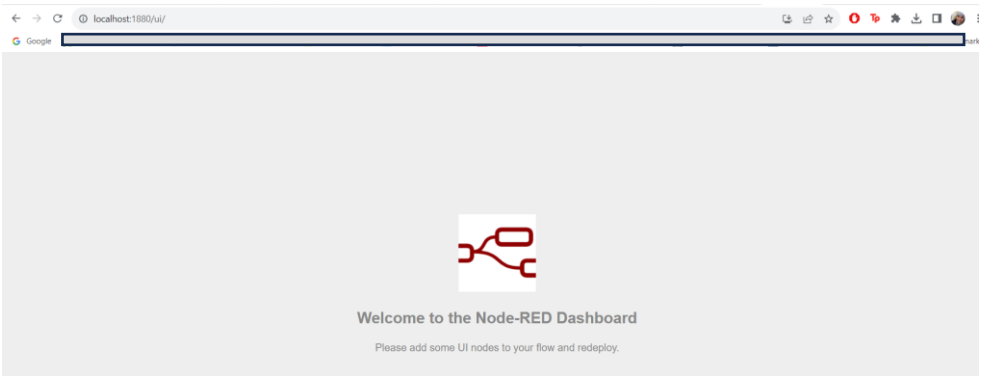
Pour la création d'un Dashboard, on a besoin de nœuds supplémentaires.

- Sélectionnez « Gérer la palette ».
- Dans l'onglet « Installer », recherchez « Dashboard ».
- Sélectionnez « node-red dashboard » et cliquez sur « install ».

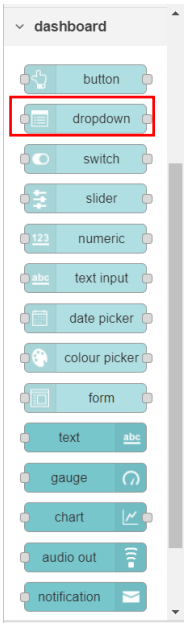
Les nœuds associés sont ajoutés à la palette afin de permettre la création d'une interface Web.

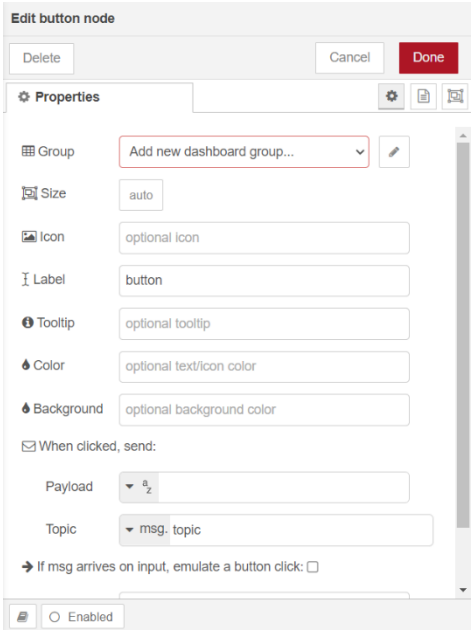


Le résultat est accessible via l'adresse url : **http://localhost:1880/ui/** (la même adresse url pour l'application Node-RED suivie de « ui » pour interface utilisateur).

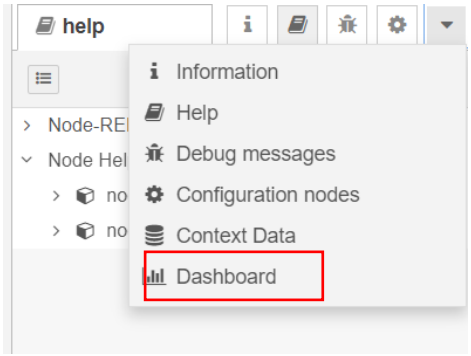


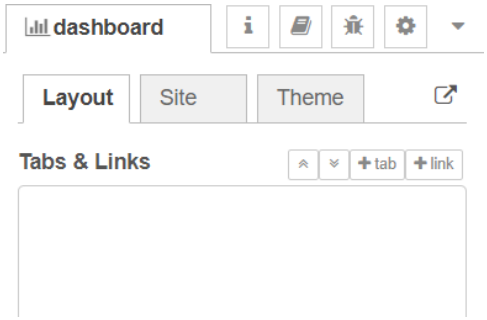
- Ajoutez le nœud du « **boutton** » sur l'espace de travail depuis la palette (Dashboard) pour construire le flux.
- Double-cliquez sur le bouton pour modifier les propriétés.



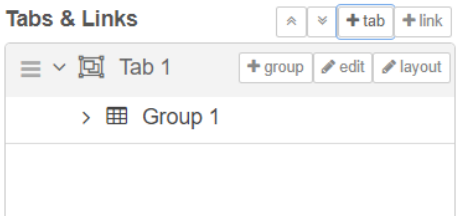


Le groupe du Dashboard sert à l'organisation de la page Web.
Le bouton « Dashboard » dans la barre latérale est utilisé pour organiser l'interface web en différentes pages (tabs) et groupes.

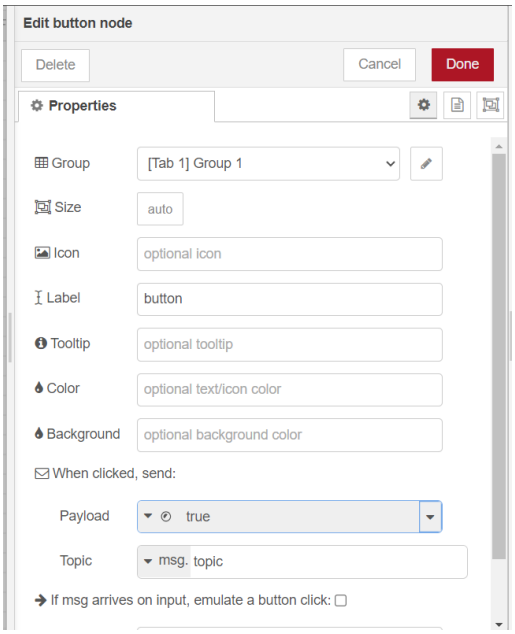




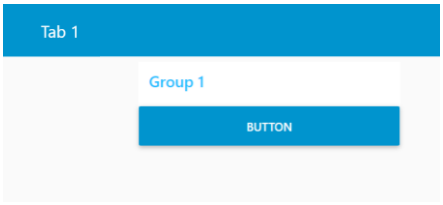
- Cliquez sur +tab pour ajouter une nouvelle page (tab).
- Ensuite, cliquez sur +group pour ajouter un nouveau groupe.



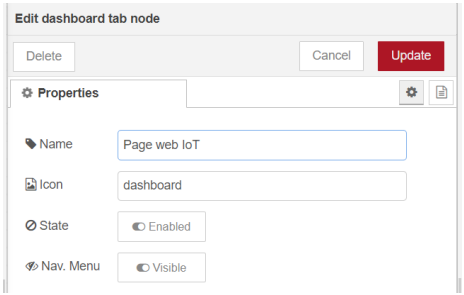
- Une fois tab1 et group1 créés, cliquez sur le bouton « Properties » dans l'espace de travail du flux et sélectionnez tab1 et group1 dans « Group ».
- Sélectionnez « Boolean » dans la propriété « Payload », c'est-à-dire que lorsqu'on clique sur le bouton, un message booléen « True » est généré.
- Cliquez sur « Done » puis « Deploy ».

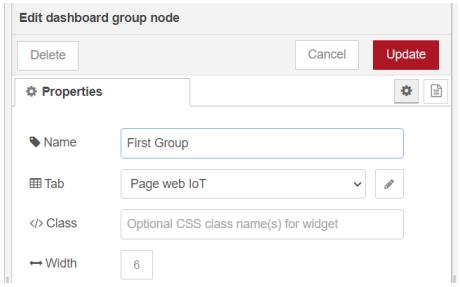


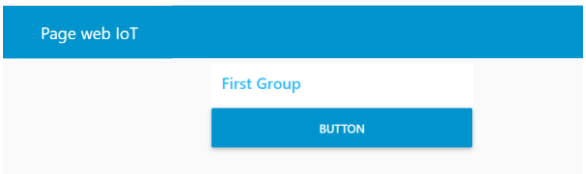
Vous devez avoir le résultat suivant.



- Modifiez la propriété Nom de page (tab).
- Cliquez sur « update ».
- Modifiez la propriété Nom de groupe.
- Cliquez sur « update ».
- Déployez à nouveau et vérifiez la modification effectuée sur la page Web.







- Cliquez sur le bouton dans votre interface web.

Q7 : Expliquez les résultats obtenus dans la fenêtre de débogage.

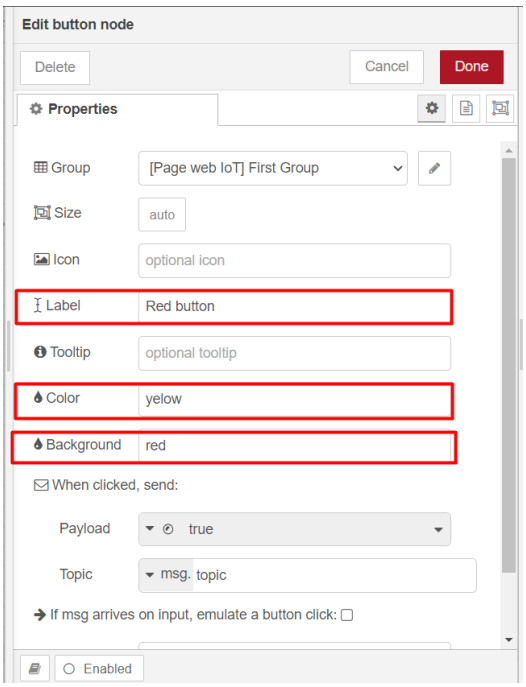
- Ajoutez un nœud de « **debug** » dans le flux.



- Sélectionnez la fenêtre de « debug » dans la barre latérale de l'éditeur Node-RED.
- Déployez à nouveau.

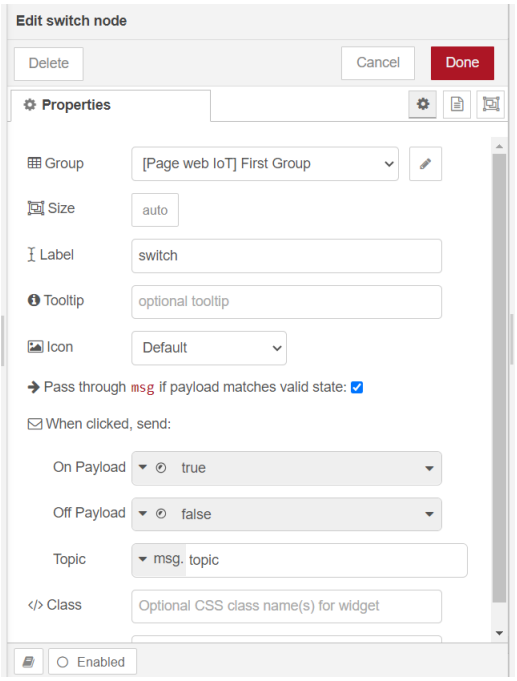
Q8 : Cliquez sur le bouton dans le web et commentez les résultats obtenus dans la fenêtre de débogage.

- Modifiez les propriétés du bouton pour changer les couleurs et le nom dans « label ».

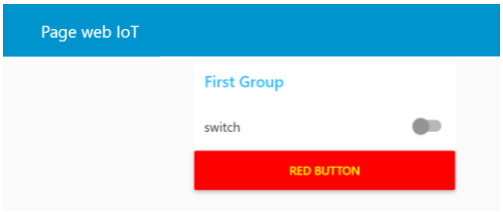


- Cliquez sur « Done » puis « Deploy », la page web est mise à jour.
- Cliquez sur « Dashboard » dans la barre latérale pour modifier l'apparence de la page Web.
- Cliquez sur « Site » et « Theme » et modifiez les apparences selon votre choix.
- Déployez à nouveau pour mettre à jour les modifications.

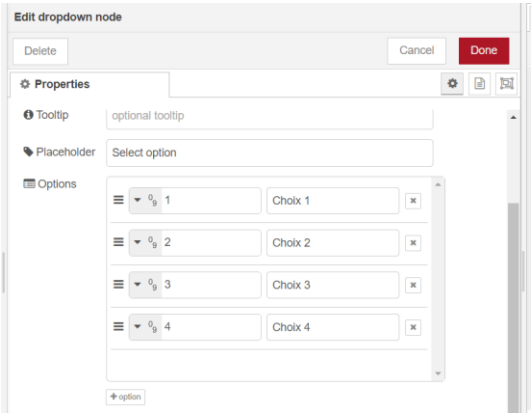
- Ajoutez le nœud « **switch** » sur l'espace de travail depuis la palette (Dashboard). Le nœud switch a deux états stables : « On payload » et « Off payload ». Conservez les valeurs par défaut (Vrai et Faux).



- Cliquez sur « Done ».
- Connectez le nœud de switch au nœud debug.
- Cliquez sur « Deploy ».
- Cliquez sur le bouton switch dans votre Dashboard.
  Q9 : Expliquez les résultats obtenus dans la fenêtre de débogage.



- Ajoutez le nœud « **dropdown** » sur l'espace de travail depuis la palette de Dashboard.
- Modifiez les propriétés du bouton (options) comme suit :

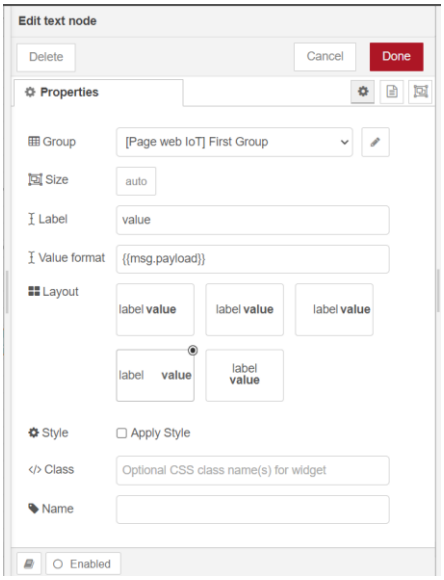


- Cliquez sur « Done ».
- Connectez le nœud « dropdown » au nœud debug.
- Déployez et voyez les modifications dans l'interface de la page Web.
- Sélectionnez les choix un à un.

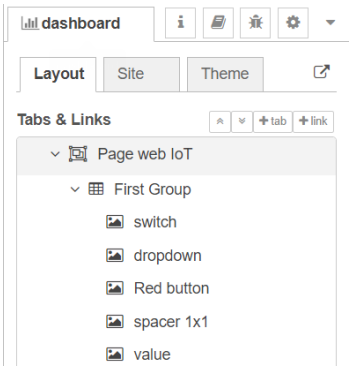
Q10 : Expliquez les résultats obtenus dans la fenêtre de débogage.

Tous les nœuds utilisés permettent de représenter les entrées depuis l'interface de la page Web. Nous allons maintenant présenter les nœuds qui permettent de représenter une sortie vers l'interface.

- Ajoutez le nœud « **text** » sur l'espace de travail depuis la palette Dashboard.
- Modifiez les propriétés du nœud en changeant le label en « value ».
- Cliquez sur « Done » puis connectez ce nœud de texte au nœud « dropdown ».

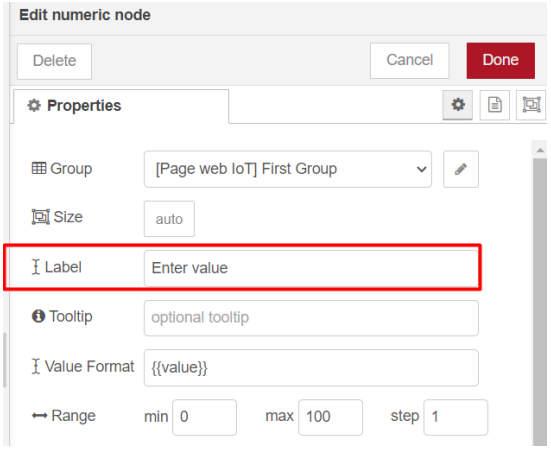


- Editez la vue texte dans l'interface pour occuper la dernière position (faites glisser value dans la fenêtre Tab & Links) et ajoutez un espace pour la séparer des boutons (+spacer).

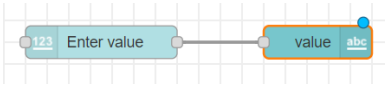


Q11 : Déployez, vérifiez l'affichage de la page web et expliquez les résultats obtenus.

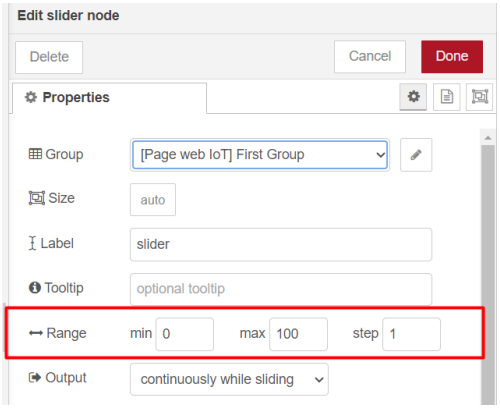
- Ajoutez le nœud « **numeric** » et modifiez les propriétés du nœud comme suit puis cliquez sur « Done ».



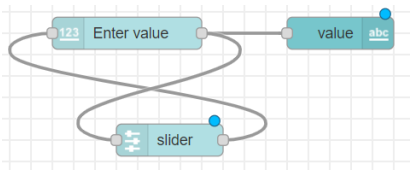
- Connectez ces nœuds comme suit :



- Déployez et vérifiez les résultats dans l'interface.
- Entrez différentes valeurs et vérifiez les résultats.
- Ajoutez le nœud « **slider** ».
- Modifiez les propriétés du nœud comme suit, cliquez sur « Done ».

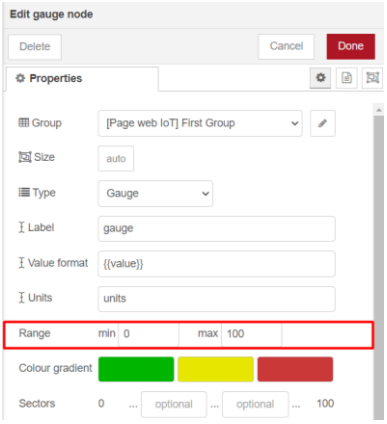


- Connectez le nœud slider au nœud de valeur.
- Déployez et vérifiez les résultats dans l'interface.
- Reproduisez le flux suivant.

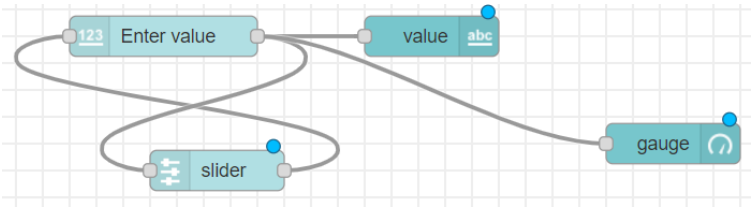


Q12 : Commentez les résultats obtenus dans l'interface.

- Ajoutez le nœud de « **gauge** ».
- Modifiez les propriétés du nœud comme suit, cliquez sur « Done ».



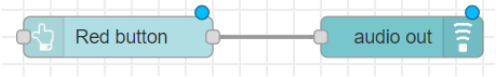
- Connectez le nœud Enter Value au nœud de gauge.



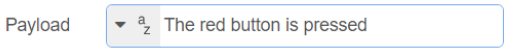
- Déployez et vérifiez les résultats dans l'interface.
- Afin de mieux organiser l'interface, créez un deuxième groupe dans « Tab&Links ».
- Modifiez le nœud de gauge pour qu'il soit dans le deuxième groupe.
- Déployez et vérifiez les résultats dans l'interface.
- Ajoutez le nœud « **chart** ».
- Connectez-le au nœud Enter Value
- Conservez les propriétés par défaut et placez-le dans la catégorie du deuxième groupe.

Q13 : Déployez et commentez les résultats obtenus dans l'interface.

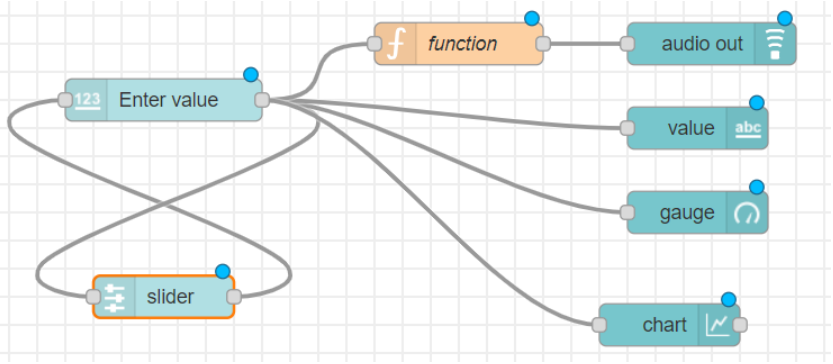
- Ajoutez le nœud « **audio** ».
- Reproduisez le flux suivant.



- Modifiez la propriété du Red button pour générer « string payload ».

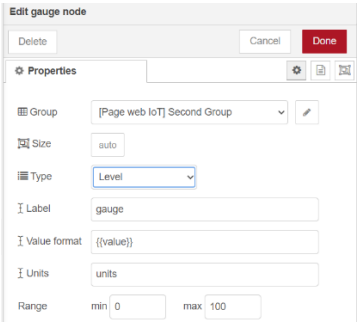


- Appuyez sur Red button de l'interface de la page web et vérifiez le son.
- Reproduisez le flux suivant.
- Modifiez le nœud de fonction comme suit :





- Modifiez le nœud de gauge comme suit :



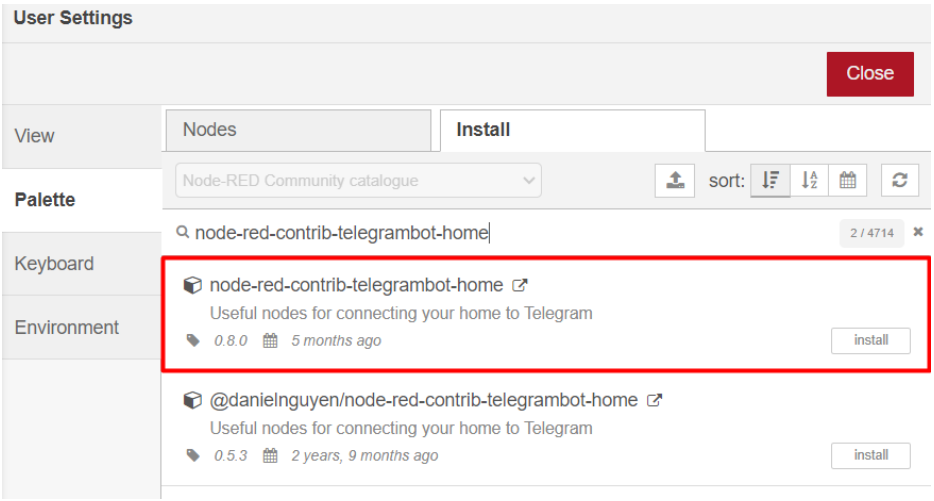
- Ajoutez un nouveau tab et ajoutez quelques boutons et sorties.

III. Création de flux de communication utilisant Telegram

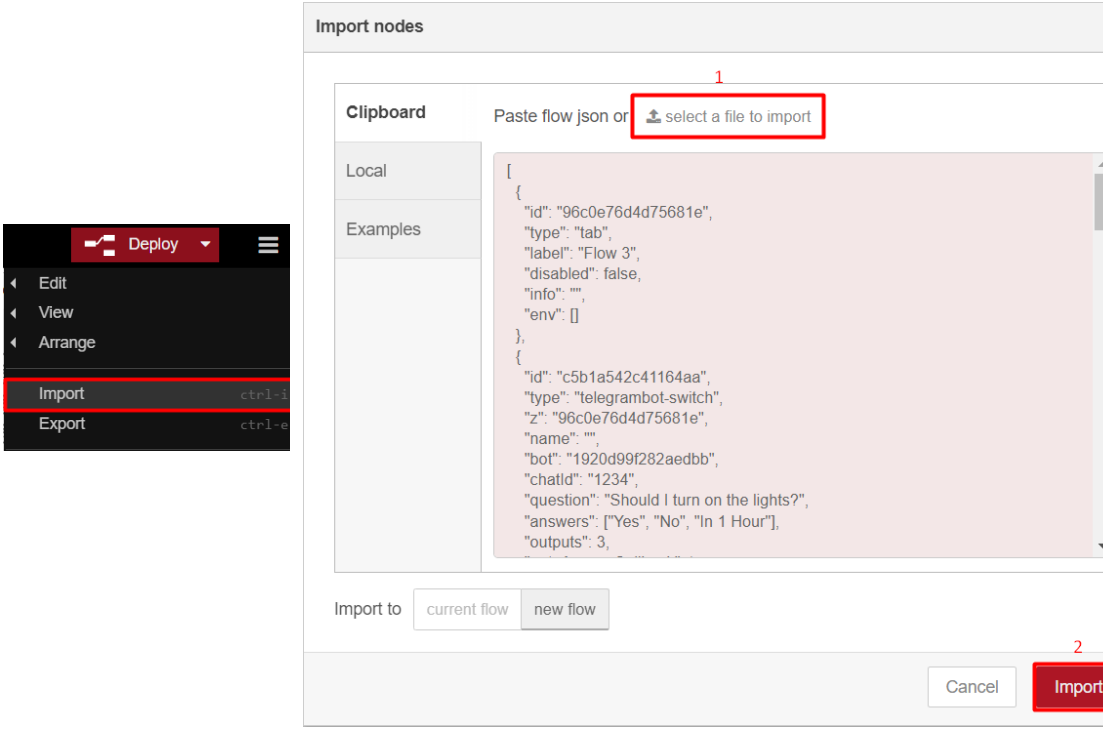
Pour assurer une communication avec les utilisateurs de l'application IoT, il est possible d'utiliser des nœuds de communication. Dans ce TP, vous allez explorer les nœuds de Telegram.

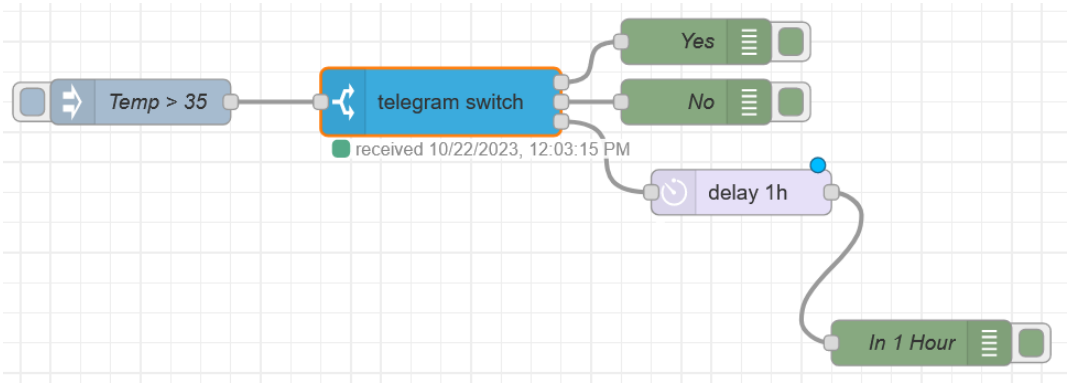
- Sélectionnez « Gérer la palette ».
- Dans l'onglet « Installer », recherchez « node-red-contrib-telegrambot-home ».
- Sélectionnez « install ».

Les nœuds associés sont ajoutés à la palette.



- Téléchargez le fichier switch.json.
- Utilisez les instructions d'importation/exportation pour importer le flux ci-dessous.

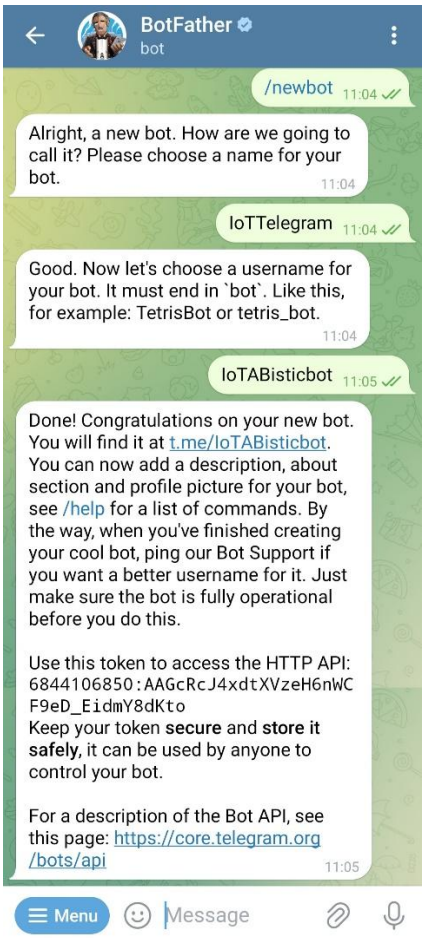




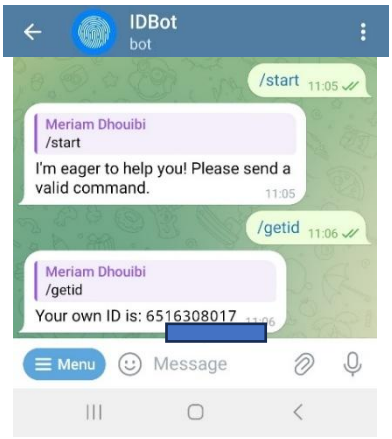
1. Configuration du bot Telegram

Vous devrez configurer un bot **Telegram** pour utiliser ces nœuds. Vous pouvez suivre ces instructions pour savoir comment procéder.

- Ouvrez Telegram et cherchez BotFather
- Envoyez à BotFather le message « /newbot » pour créer le bot :
- Nommez le bot : **n'importe quel nom**.
- Donnez un nom d'utilisateur au bot : **doit être unique**.
- Récupérez le jeton (**token)** : vous en aurez besoin pour configurer les nœuds.

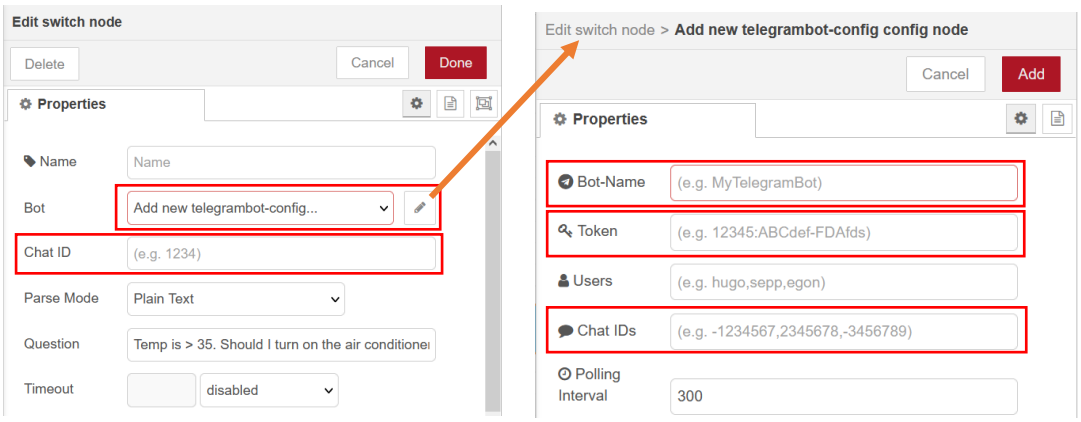


- Cherchez « IDBot »
- Envoyez à « IDBot » le message « /getid » pour récupérer l'ID de chat.

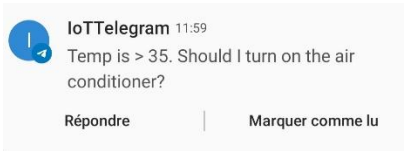


2. Configuration des nœuds de Telegram

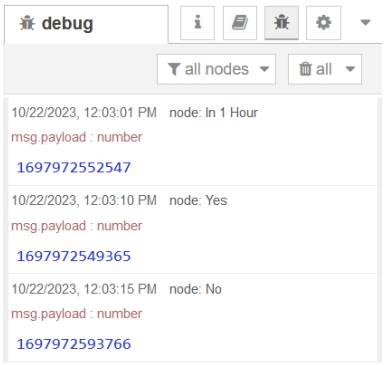
- Double-cliquez sur le nœud « **telegram switch** » dans votre flux et modifiez les propriétés comme suit :



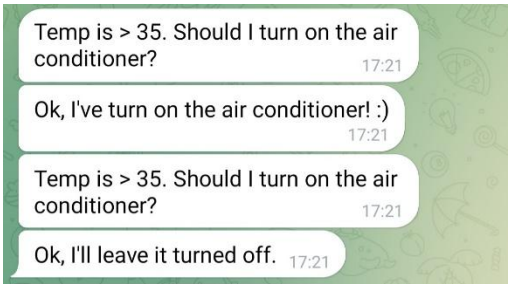
- Insérez votre Bot-Name, Token et ID de chat.
- Cliquez sur déployer.
- Vous devez ainsi avoir les résultats suivants.







Q14 : Proposez un flux pour communiquer avec l'utilisateur en lui envoyant une confirmation de l'action. Exemple :



Utilisez le nœud « **telegram notify** ».

TP4 – Mise en œuvre du protocole MQTT avec Node-RED pour la visualisation des données avec ESP32 et le capteur DHT11

Introduction

Dans ce TP, vous allez explorer Node-RED et MQTT pour visualiser les valeurs issues du capteur DHT11 sur Dashboard.

Objectifs

À la fin de ce TP, vous serez en mesure de :

- Utiliser mosquitto broker pour publish/subscribe des messages.
- Constituer un client MQTT qui écoute plusieurs topics.
- Utiliser MQTT et Node-RED pour la réalisation des applications IoT.

Manipulation

I. MQTT sous Windows

1. Installation de mosquitto broker

- Téléchargez le broker mosquitto approprié pour Windows depuis la page Web . (Conservez les paramètres d'installation par défaut)

- Une fois installé, ouvrez une invite de commande et changez de répertoire pour le dossier de destination mosquito.

```
C:\Users\pc>cd C:\Program Files\mosquitto

C:\Program Files\mosquitto>
```

- Ensuite, exécutez moquitto.exe depuis l'invite de commande.

```
C:\Program Files\mosquitto>mosquitto.exe
```

2. Test de mosquitto broker en local

On vous propose d'utiliser localement le broker mosquitto pour tester le flux suivant :



Le capteur publie le topic temperature. Un équipement distant doit s'abonner pour obtenir ces informations.

- Le broker attend alors les informations du capteur pour les publier (vous l'avez déjà lancé sur cmd).

- Ouvrez un nouveau cmd **pour imiter l'équipement d'un abonné distant**.
- Exécutez la commande suivante pour s'abonner au topic « temperature » :

```
C:\Program Files\mosquitto>mosquitto_sub -h localhost -t temperature
```

  - -h : mqtt host auquel se connecter. La valeur par défaut est localhost, cela signifie, sur quel broker passer cette commande.
  - -t : mqtt topic sur lequel s'abonner (temperature).

- Maintenant, vous devez **simuler le comportement du capteur** à l'aide d'un nouveau cmd.

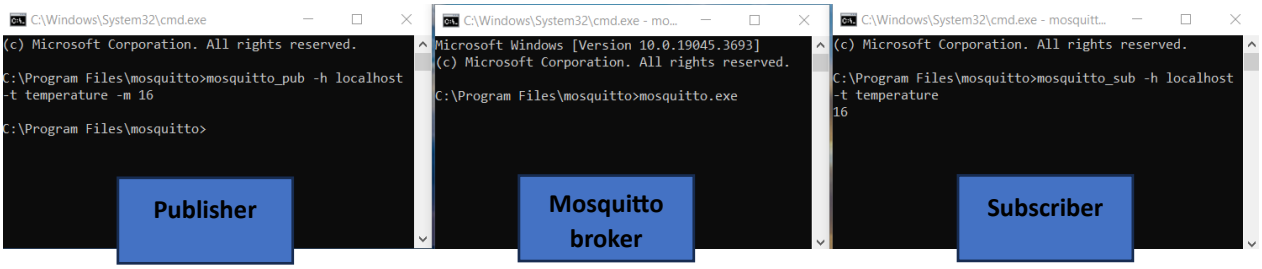
- Exécutez la commande suivante pour publier la valeur 16°C pour la température à l'aide du serveur localhost.

```
C:\Program Files\mosquitto>mosquitto_pub -h localhost -t temperature -m 16

C:\Program Files\mosquitto>
```

  - -t : mqtt topic sur lequel publier (temperature).
  - -m : message payload à envoyer : le payload (temperature) value

Le broker reçoit le message et peut publier ces informations aux abonnés.



Q1 : Publier différentes valeurs de température.

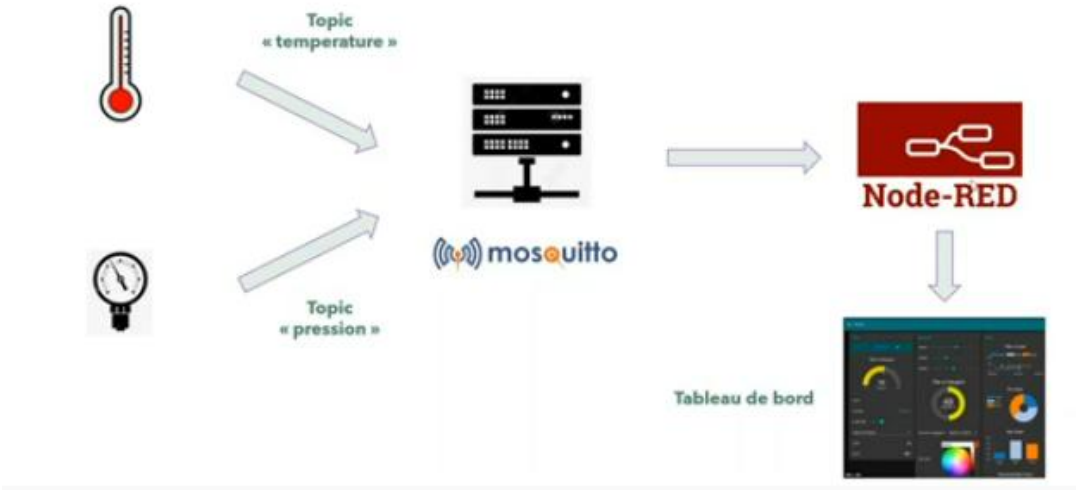
Q2 : Reproduire cet exemple avec 2 capteurs ou plus et 2 équipements ou plus.

Remarque : L'équipement distant peut s'abonner à plusieurs Topics (exemple : température et pression).

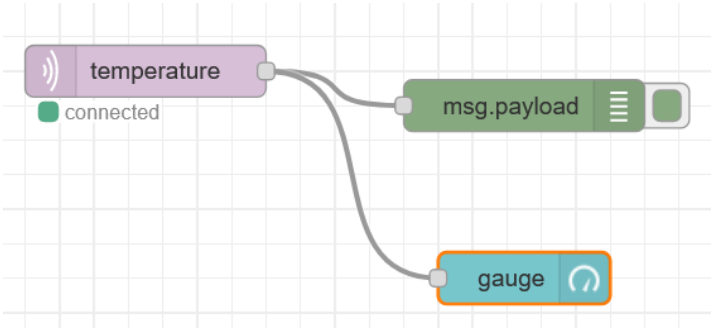
II. MQTT et Node-RED

1. Node-RED Subscriber

Dans cette configuration, l'équipement distant sera l'application Node-RED qui est abonnée aux topics « temperature » et « pression ». Le Dashboard dans Node-RED, permet de voir les résultats de manière très illustrative.

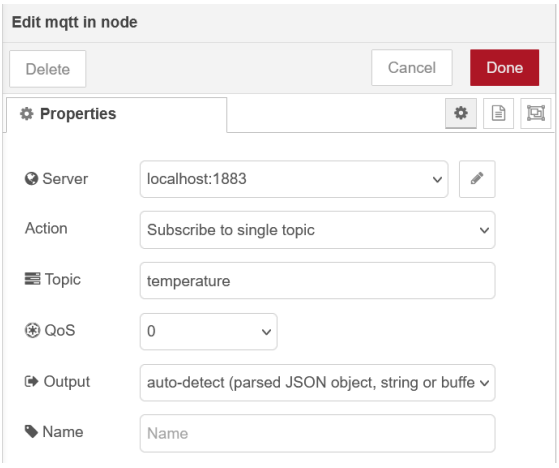


- Simulez les capteurs de température et de pression.
- Exécutez le mosquitto broker
- Reproduisez le flux suivant avec l'application Node-RED exécutée localement :



Configurations :

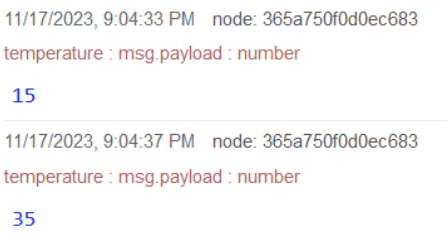
- Nœud MQTT



- Ouvrez un nouveau cmd et exécutez mosquitto puis publiez une valeur de température et observez les résultats dans la fenêtre de débogage de Node-RED.

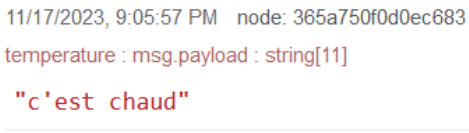
```
C:\Program Files\mosquitto>mosquitto_pub -h localhost -t temperature -m 15

C:\Program Files\mosquitto>mosquitto_pub -h localhost -t temperature -m 35
```

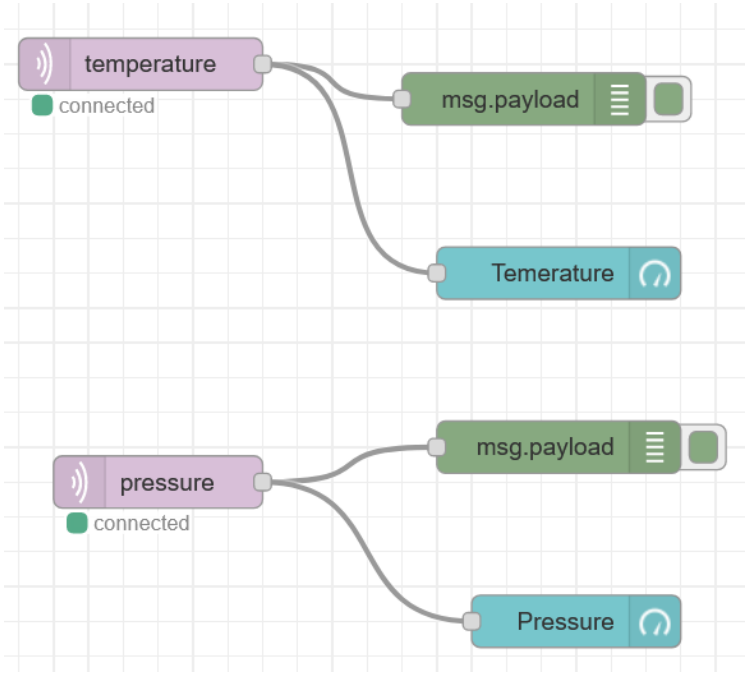


Le message publié peut-être aussi une chaîne de caractère.

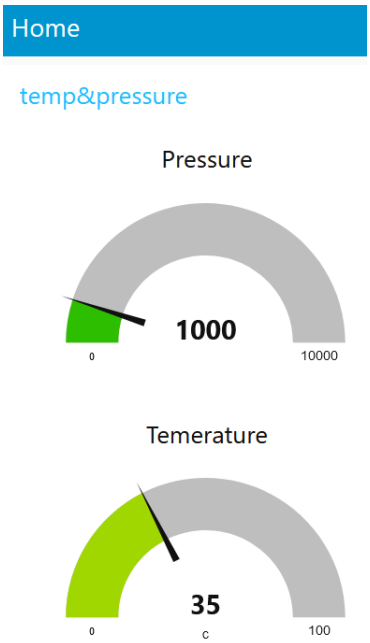
```
C:\Program Files\mosquitto>mosquitto_pub -h localhost -t temperature -m "c'est chaud"
```



- Complétez le flux avec le capteur de pression comme suit :



- Configurez les nœuds des gauges pour afficher les résultats obtenus sur Dashboard (http://localhost:1880/ui/).



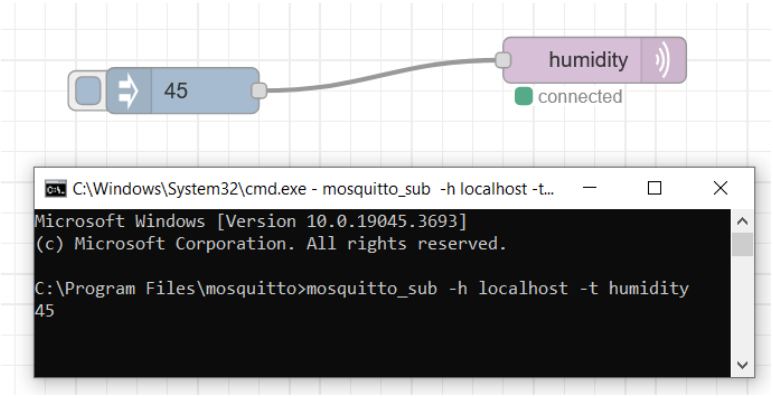
2. Node-RED Publisher

Node-RED peut également être utilisé pour publier les informations d'humidité du capteur par exemple.

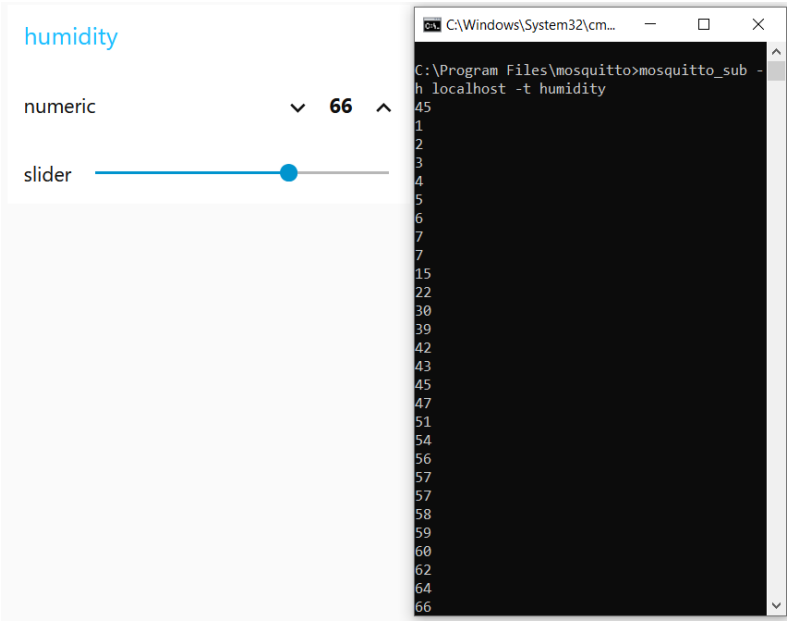
- Créez un équipement abonné :

```
C:\Program Files\mosquitto>mosquitto_sub -h localhost -t humidity
```

- Réalisez le flux suivant, déployez, injectez l'entrée et consultez les résultats dans l'invite de commande de l'abonné.

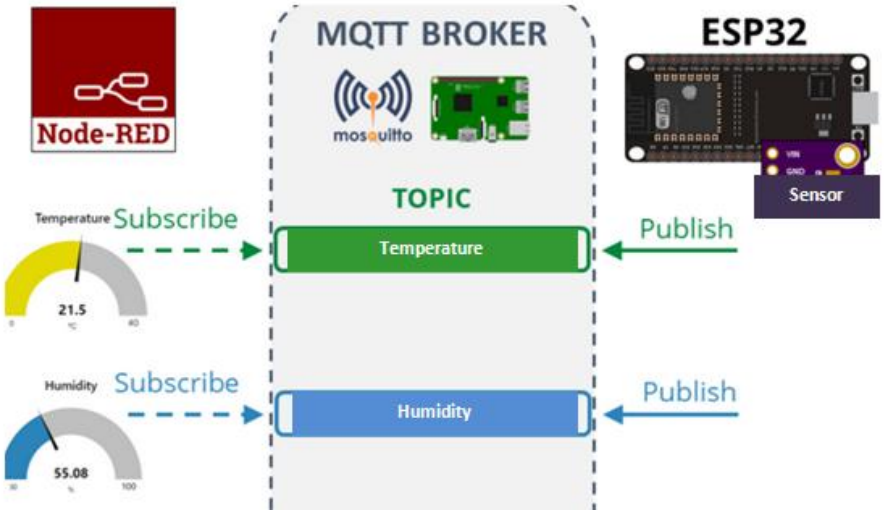


- Reproduisez le flux suivant. Déployez, modifiez les valeurs d'humidité à partir de Dashboard et consultez les résultats dans l'invite de commande de l'abonné.



III. Application IoT : Node-RED, MQTT, ESP32 et DHT11

Le diagramme suivant montre l'aperçu de l'application :



- Matériels utilisés :
  - ESP32
  - Capteur DHT11
  - PC
- Logiciels et services en ligne
  - Arduino IDE
  - Node-RED
  - Mosquitto broker

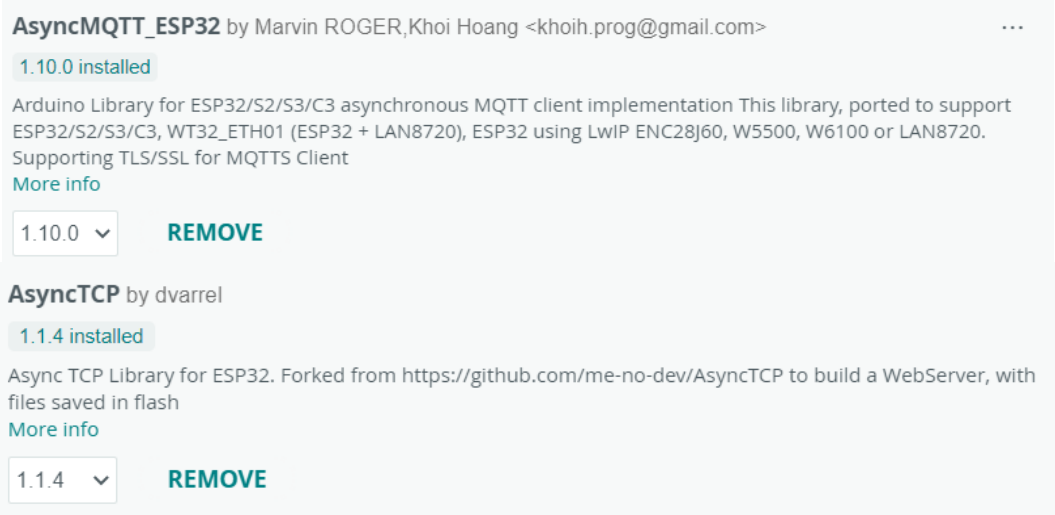
1. Configuration de l'Arduino IDE

Vous aurez besoin de l'IDE Arduino et de tous les logiciels et bibliothèques associés installés.

- La prise en charge de la carte ESP32 doit être ajoutée à l'IDE Arduino avant d'écrire le programme et le flasher dans la carte. Pour ce faire, installez-le package ESP32 (Déjà installé dans le TP4).

Pour simplifier le développement d'applications, installez les bibliothèques à partir du référentiel de bibliothèque Arduino standard :

- Cliquez sur « Sketch » puis « Include Library ». Sélectionnez « Manage Libraries ».
- Recherchez et installez les bibliothèques **Async MQTT Client**, **Async TCP** et **DHT Sensor Library for ESPx (TP4)**.



2. Préparation du code

1- Vous devez modifier les constantes et variables suivantes dans le code.

- WIFI_SSID
- WIFI_PASSWORD
- MQTT_HOST : test.mosquitto.org
- MQTT_PORT : 1883
- MQTT_PUB_TEMP : esp32/dht/temperature
- MQTT_PUB_HUM : esp32/dht/humidity

2- Précisez le pin connecté au capteur DHTPIN (pin DATA).
3- Copiez le code, le compilez et le téléversez.

Code

```
#include "DHT.h"
#include <WiFi.h>
#include <AsyncMqttClient.h>

#define WIFI_SSID "*****"
#define WIFI_PASSWORD "******"
```

```
#define MQTT_HOST "******"
#define MQTT_PORT ****
// MQTT topics
#define MQTT_PUB_TEMP "******"
#define MQTT_PUB_HUM "*******"
// DHT sensor configuration
#define DHTPIN *  // DHT sensor connected to pin 4
#define DHTTYPE DHT11  // Change to DHT22 if needed
DHT dht(DHTPIN, DHTTYPE);

// MQTT client
AsyncMqttClient mqttClient;
void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  // Wait for Wi-Fi connection
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Wi-Fi connected");
  mqttClient.onConnect([](bool sessionPresent) {
    Serial.println("Connected to MQTT broker");
    mqttClient.publish(MQTT_PUB_TEMP, 1, true, "Connected");
    mqttClient.publish(MQTT_PUB_HUM, 1, true, "Connected");
  });

  mqttClient.onDisconnect([](AsyncMqttClientDisconnectReason reason) {
    Serial.println("Disconnected from MQTT broker");
    if (WiFi.isConnected()) {
      Serial.println("Attempting to reconnect to MQTT broker...");
      mqttClient.connect();
    }
  });
  mqttClient.setServer(MQTT_HOST, MQTT_PORT);
  mqttClient.connect();
}
void loop() {
  static unsigned long previousMillis = 0;
  unsigned long currentMillis = millis();
  const long interval = 10000;  // Set the interval for publishing

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

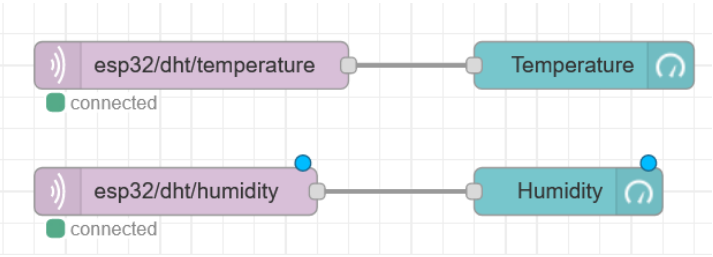
    // Read humidity and temperature from the DHT sensor
```

```
    float hum = dht.readHumidity();
    float temp = dht.readTemperature();

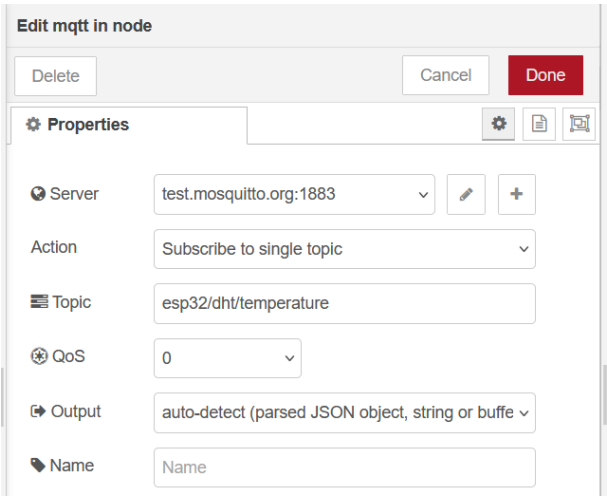
    if (isnan(temp) || isnan(hum)) {
      Serial.println("Failed to read from DHT sensor!");
      return;
    }
    // Print sensor values for debugging
    Serial.print("Temperature: ");
    Serial.println(temp);
    Serial.print("Humidity: ");
    Serial.println(hum);
    // Publish temperature and humidity to MQTT broker
    Serial.println("Publishing temperature...");
    mqttClient.publish(MQTT_PUB_TEMP, 1, true, String(temp).c_str());
    Serial.println("Publishing humidity...");
    mqttClient.publish(MQTT_PUB_HUM, 1, true, String(hum).c_str());
  }
}
```

3. Preparation de Node-RED flux et Dashboard:

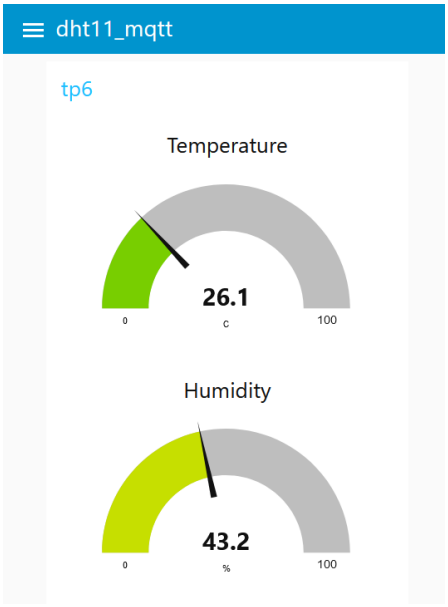
- Ouvrez Node-RED
- Ajoutez deux nœuds MQTT et deux nœuds gauge de Dashboard au flux.



Mqtt nœud configuration :



- Enfin, déployez votre flux.
- Vous devriez avoir accès aux relevés actuels de température et d'humidité issues du capteur DHT11 sur votre Dashboard. Vous pouvez utiliser d'autres nœuds de Dashboard pour afficher les relevés de différentes manières.



TP5 – Communication HTTP avec Node-RED et ESP32 : implémentation des requêtes GET/POST et traitement des données

Introduction

Dans ce TP, vous allez découvrir la communication HTTP avec Node-RED puis entre un ESP32 et Node-RED. Vous apprendrez à utiliser Node-RED pour échanger des données via des requêtes HTTP et à exploiter ces données dans des applications IoT.

Objectifs

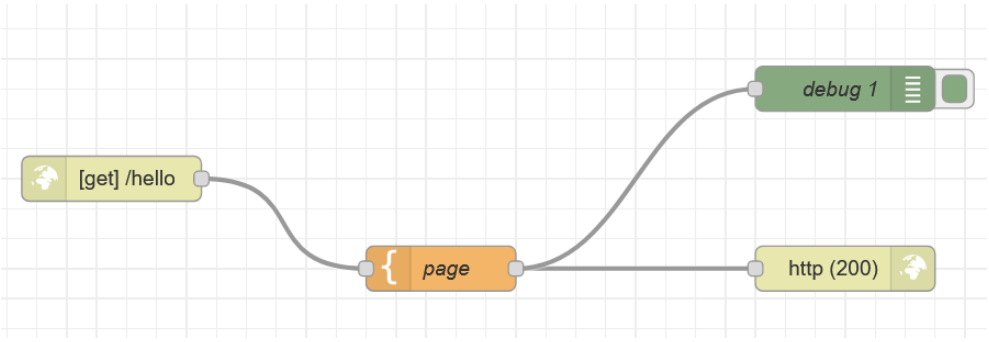
À la fin de ce TP, vous serez en mesure de :

- Implémenter des requêtes HTTP GET et POST avec l'ESP32.
- Traiter et visualiser les données échangées.
- Utiliser MQTT et HTTP pour la réalisation des applications IoT.

Manipulation

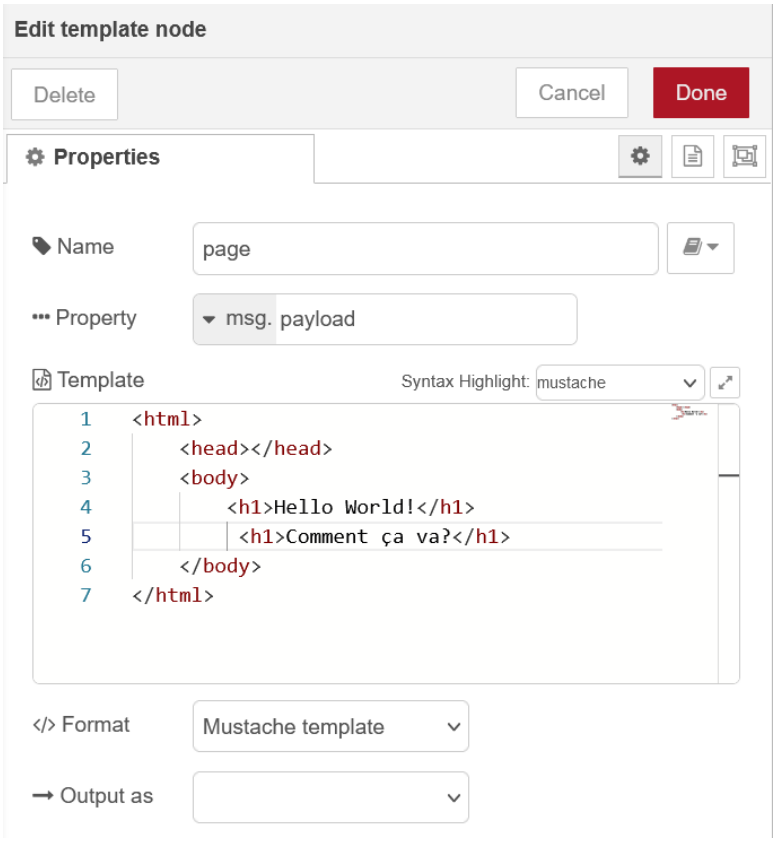
I. Http hello

- Reproduisez le flux suivant avec l'application Node-RED exécutée localement :

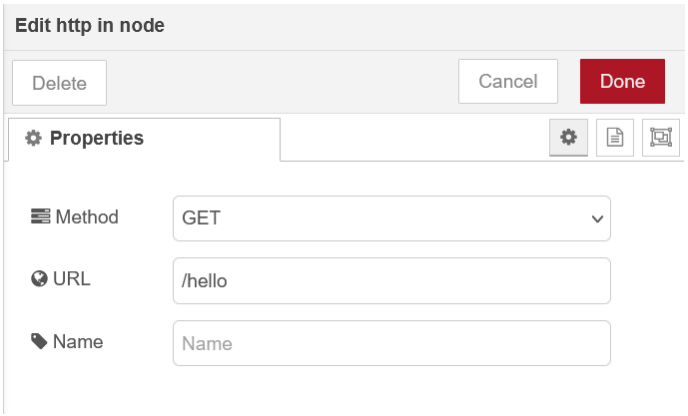


Configurations :

- Nœud template



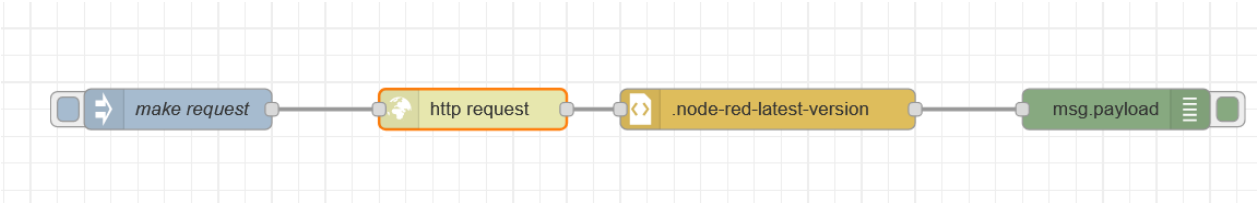
- Nœud http in



- Déployez
- Ouvrez le lien suivant : http://localhost:1880/hello
- Expliquez les résultats obtenus.

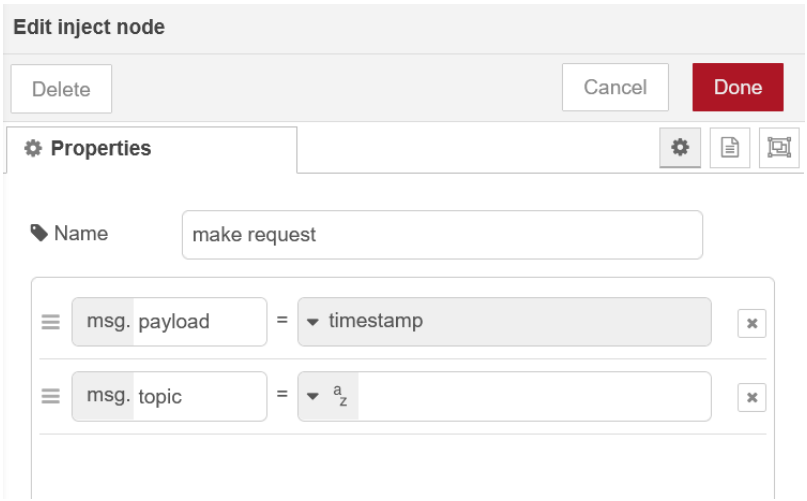
II. Http « Get »

- Reproduisez le flux suivant :

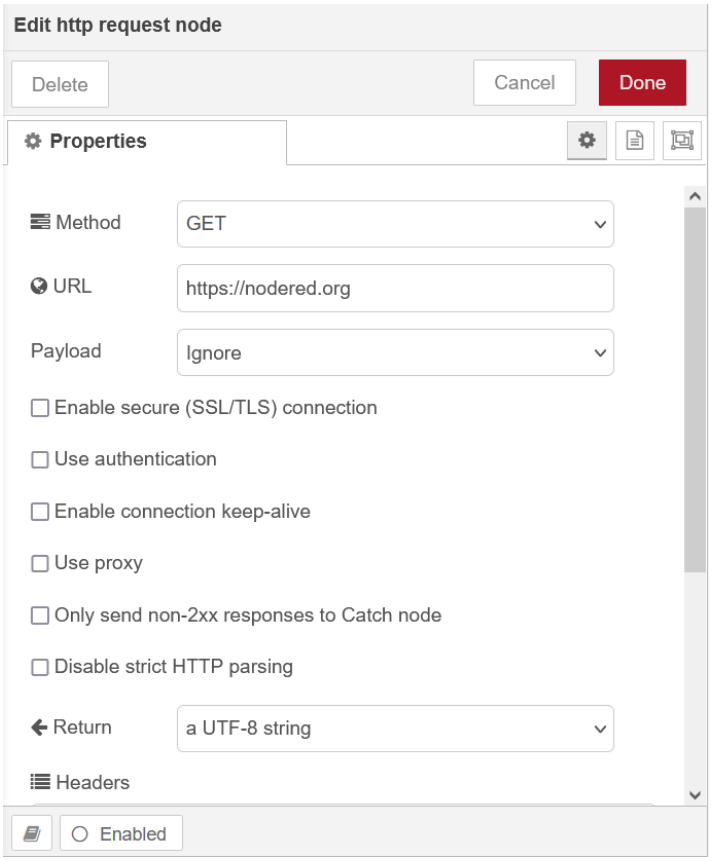


Configurations :

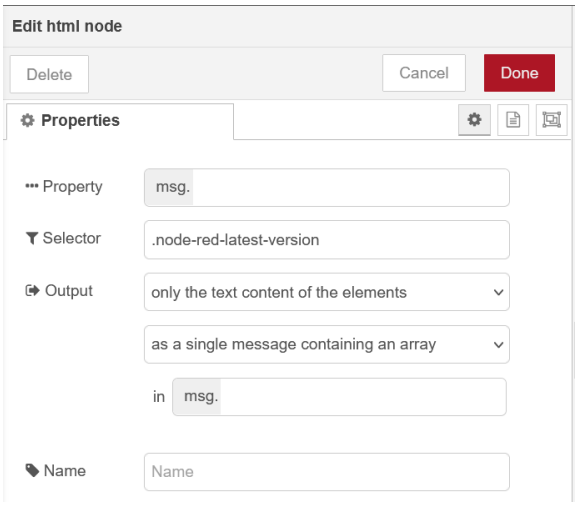
- Nœud inject



- Nœud http request



- Nœud html

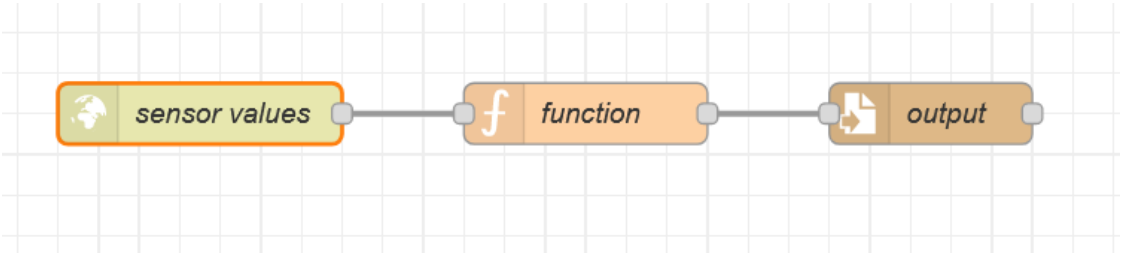


- Déployez
- Expliquez les résultats obtenus sur la fenêtre de débogage.

III. Http « Get sensor values »

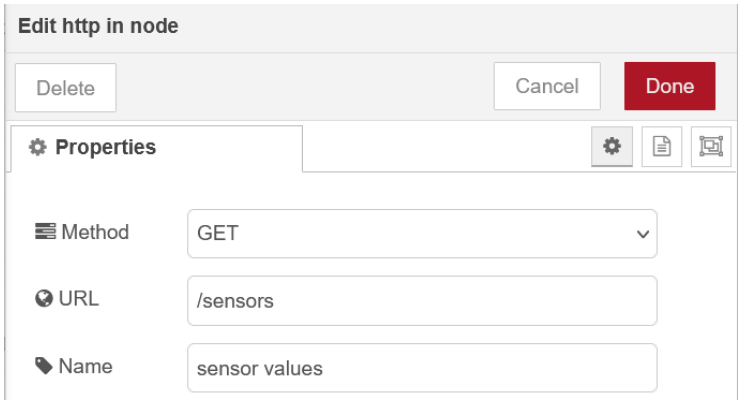
Vous allez recevoir les valeurs de la requête http et enregistrer-les dans un fichier.

- Reproduisez le flux suivant :

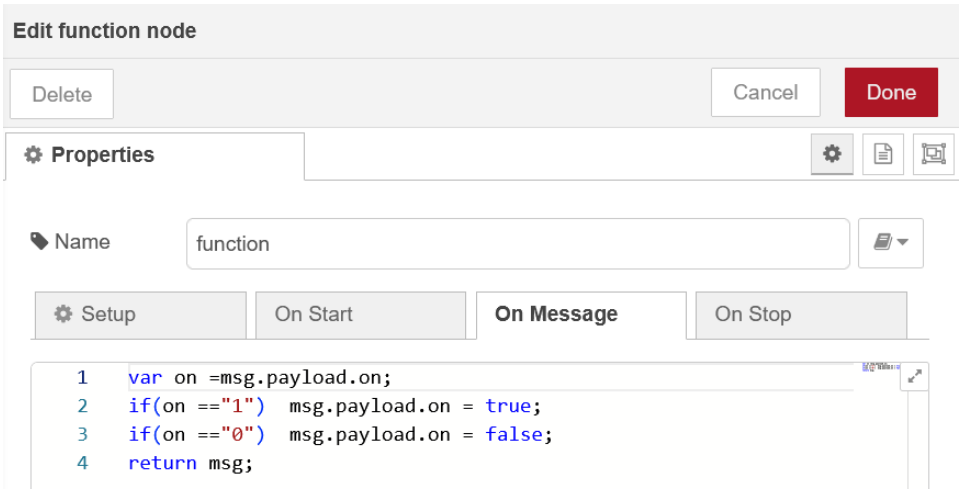


Configurations :

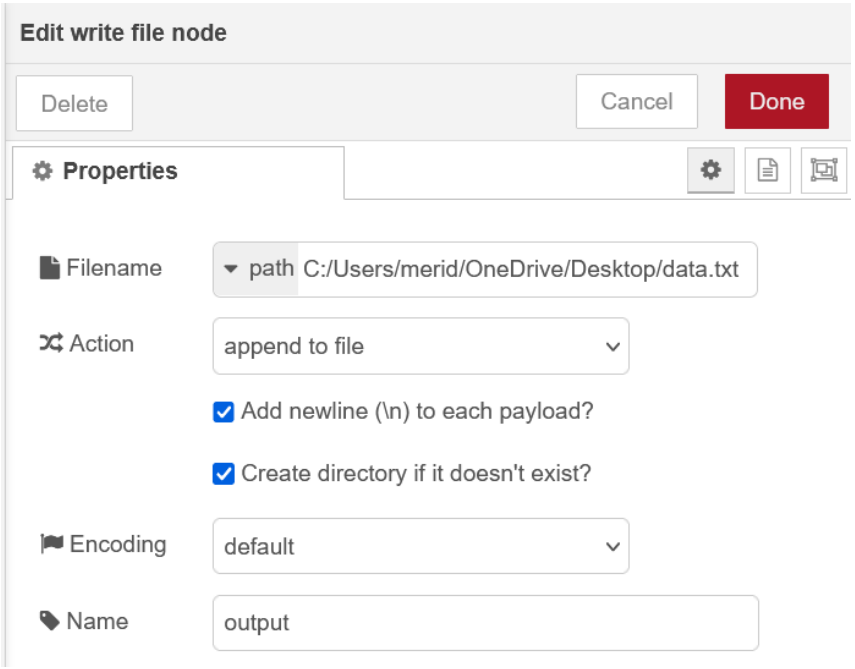
- Nœud http in



- Nœud function



- Nœud write file



- Déployez
- Ouvrez le lien suivant : http://localhost:1880/sensors?temp=20&hum=45&on=1
- Ouvrez le fichier « data.txt »
- Expliquez les résultats obtenus.

IV. Http & Node-RED

1. Configuration Node-RED

- Ouvrez le fichier « settings.js » qui se trouve sous le userDir dans ~/.node-red et décommentez la ligne uiHost et modifiez l'adresse IP selon l'adresse IP statique de votre machine hôte.

```
141  ********************************************************************************/
142
143      /** the tcp port that the Node-RED web server is listening on */
144      uiPort: process.env.PORT || 1880,
145
146      /** By default, the Node-RED UI accepts connections on all IPv4 interfaces.
147       * To listen on all IPv6 addresses, set uiHost to "::",
148       * The following property can be used to listen on a specific interface. For
149       * example, the following would only allow connections from the local machine.
150       */
151      //uiHost: "127.0.0.1",
152
```

Exemple :

```
/** the tcp port that the Node-RED web server is listening on */
uiPort: process.env.PORT || 1880,

/** By default, the Node-RED UI accepts connections on all IPv4 interfaces.
 * To listen on all IPv6 addresses, set uiHost to "::",
 * The following property can be used to listen on a specific interface. For
 * example, the following would only allow connections from the local machine.
 */
uiHost: "192.168.1.21",
```

- Ouvrez un cmd et lancez Node-RED
- Importez le flux suivant et cliquez sur déployer.

2. Préparation du code1

1- Vous devez modifier les constantes et variables suivantes dans le code1.

- ssid
- password
- serverName (votre IP address)

2- Copiez le code1, le compilez et le téléversez.

Code1

```
#include <WiFi.h>
#include <HTTPClient.h>
const char* ssid = "**********";
const char* password = "********";
//Your Domain name with URL path or IP address with path
String serverName = "http://*****:1880/update-sensor";
```

```
unsigned long lastTime = 0;
// Timer set to 10 minutes (600000)
//unsigned long timerDelay = 600000;
// Set timer to 5 seconds (5000)
unsigned long timerDelay = 5000;
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.println("Connecting");
  while(WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.print("Connected to WiFi network with IP Address: ");
  Serial.println(WiFi.localIP());

  Serial.println("Timer set to 5 seconds (timerDelay variable), it will take 5
seconds before publishing the first reading.");
}
void loop() {
  if ((millis() - lastTime) > timerDelay) {

    if(WiFi.status()== WL_CONNECTED){
      HTTPClient http;
      String serverPath = serverName + "?temperature=24.37";
      http.begin(serverPath.c_str());
      int httpResponseCode = http.GET();

      if (httpResponseCode>0) {
        Serial.print("HTTP Response code: ");
        Serial.println(httpResponseCode);
        String payload = http.getString();
        Serial.println(payload);
      }
      else {
        Serial.print("Error code: ");
        Serial.println(httpResponseCode);
      }
      http.end();
    }
    else {
      Serial.println("WiFi Disconnected");
    }
    lastTime = millis();  }}
```

- Expliquez les résultats obtenus (fenêtre de débogage et moniteur série de Arduino).

3. Préparation du code2

1- Vous devez modifier les constantes et variables suivantes dans le code2.

- ssid
- password
- serverName (votre IP address)

2- Copiez le code2, le compilez et le téléversez.

Code2

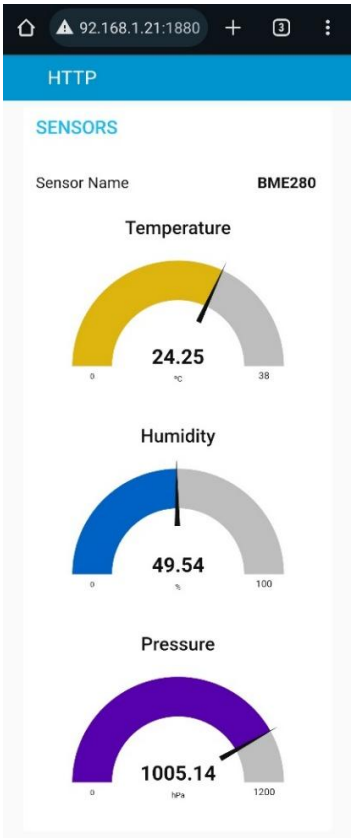
```
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "********";
const char* password = "******";
const char* serverName = "http://********:1880/update-sensor";
unsigned long lastTime = 0;
// Timer set to 10 minutes (600000)
//unsigned long timerDelay = 600000;
// Set timer to 5 seconds (5000)
unsigned long timerDelay = 5000;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.println("Connecting");
  while(WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.print("Connected to WiFi network with IP Address: ");
  Serial.println(WiFi.localIP());
  Serial.println("Timer set to 5 seconds (timerDelay variable), it will take 5
seconds before publishing the first reading.");
}
void loop() {
  //Send an HTTP POST request every 10 minutes
  if ((millis() - lastTime) > timerDelay) {
    //Check WiFi connection status
    if(WiFi.status()== WL_CONNECTED){
      WiFiClient client;
      HTTPClient http;
      http.begin(client, serverName);
      http.addHeader("Content-Type", "application/x-www-form-urlencoded");
```

```
      // Data to send with HTTP POST
      String httpRequestData =
"api_key=tPmAT5Ab3j7F9&sensor=BME280&value1=24.25&value2=49.54&value3=1005.14"
;       // Send HTTP POST request
      int httpResponseCode = http.POST(httpRequestData);
      Serial.print("HTTP Response code: ");
      Serial.println(httpResponseCode);
      http.end();
    }
    else {
      Serial.println("WiFi Disconnected");
    }
    lastTime = millis();
  }}
```

- Expliquez les résultats obtenus (fenêtre de débogage et moniteur série de Arduino).
- Ouvrez Node-Red (http://votre ip adresse:1880/) sur votre smartphone et lancez le Dashboard. Voici le résultat attendu :



TP6 – Réception et traitement des requêtes HTTP POST via Wamp et Node-RED

Introduction

Dans ce TP, vous allez étudier la réception et le traitement des requêtes HTTP POST en utilisant Wamp et Node-RED, dans le cadre d'une application IoT.

Objectifs

À la fin de ce TP, vous serez en mesure de :

- Mettre en place un serveur HTTP avec Wamp.
- Extraire et visualiser les données récupérées via Node-RED.

Manipulation

1. Node-RED

Création d'un flow Node-RED permettant de recevoir les données sur un dashboard.

1- Ouvrez le fichier « settings.js » qui se trouve sous le userDir dans ~/.node-red et décommentez la ligne uiHost et modifiez l'adresse IP selon l'adresse IP statique de votre machine hôte.

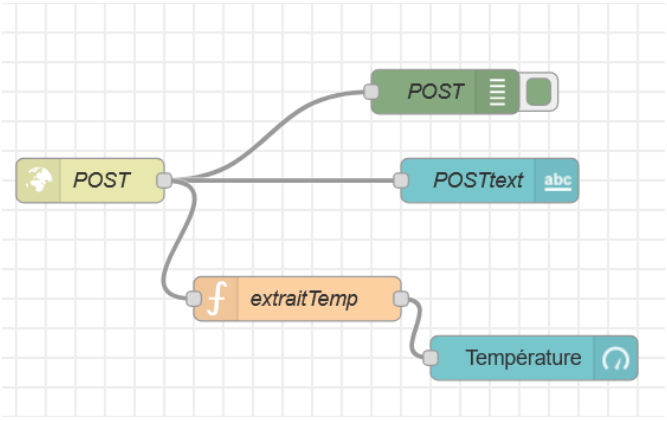
```
141  ********************************************************************************/
142
143      /** the tcp port that the Node-RED web server is listening on */
144      uiPort: process.env.PORT || 1880,
145
146      /** By default, the Node-RED UI accepts connections on all IPv4 interfaces.
147       * To listen on all IPv6 addresses, set uiHost to "::",
148       * The following property can be used to listen on a specific interface. For
149       * example, the following would only allow connections from the local machine.
150       */
151      //uiHost: "127.0.0.1",
152
```

Exemple :

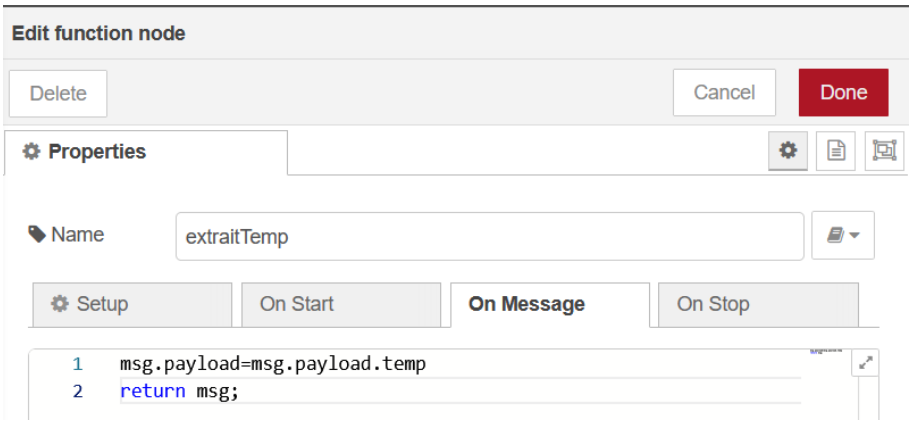
```
/** the tcp port that the Node-RED web server is listening on */
uiPort: process.env.PORT || 1880,

/** By default, the Node-RED UI accepts connections on all IPv4 interfaces.
 * To listen on all IPv6 addresses, set uiHost to "::",
 * The following property can be used to listen on a specific interface. For
 * example, the following would only allow connections from the local machine.
 */
uiHost: "192.168.1.21",
```

2- Ouvrez un cmd et lancez Node-RED
3- Réalisez le flux suivant et cliquez sur déployer.



On insère une sortie GAUGE précédée de la fonction 'extraitTemp' afin d'extraire la donnée du POST :



2. Wamp

Création d'une page html contenant un formulaire envoyant des données : température=temp et un texte=text' à Node-RED.

Attention : les "name" des <input> du formulaire sont utilisés par le POST et donc par le flux Node-RED.

1- Copiez ce code et créez la page HTML qui va être hébergée sur Wamp dans le dossier wamp\www\test.

```
<html>
<head>
<title> Envoi POST HTTP</title>
</head>
<body>
Envoi POST HTTP vers NodeRED
```

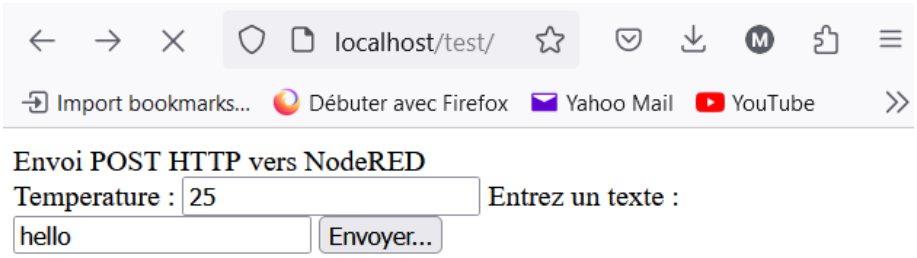
```
<form action="http://192.168.1.21:1880/essaiNODERED" method="post">
        <label for="idTemp">Temperature : </label><input id="idTemp" name="temp">
        <label for="idText">Entrez un texte : </label><input id="idText" name="texte">
        <button type="submit">Envoyer...</button>

</form>
</body>
</html>
```

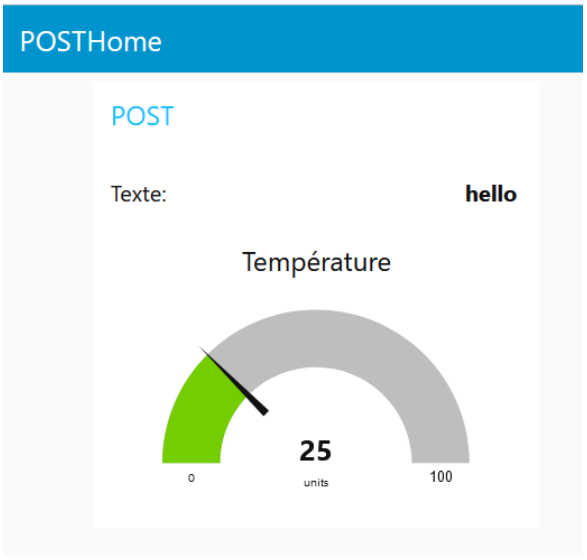
On retrouve deux entrées de formulaire :

- une pour la température (name='temp')
- une pour un texte (name='texte')

2- Lancez Wamp afin d'accèder à cette page.



3- Envoyez ces valeurs et vérifiez la réception dans votre dashboard.



Bibliographie

1. **ThingsBoard Community Edition**, lien
2. **ESP32 MQTT – Publish DHT11/DHT22 Temperature and Humidity Readings** (Arduino IDE), lien
3. **ESP32 HTTP GET and HTTP POST with Arduino IDE** (JSON, URL Encoded, Text), lien
4. **Node-RED**, lien
5. **Arduino Documentation**, lien
6. **ESP32 Datasheet**, lien
7. **ESP-IDF Programming Guide**, lien